EK-LM3S811 Firmware Development Package

USER'S GUIDE



SW-EK-LM3S811-UG-6288

Copyright © 2006-2010 Texas Instruments Incorporated.

Copyright

Copyright © 2006-2010 Texas Instruments Incorporated. All rights reserved. Stellaris and StellarisWare are registered trademarks of Texas Instruments. ARM and Thumb are registered trademarks and Cortex is a trademark of ARM Limited. Other names and brands may be claimed as the property of others.

Please be aware that an important notice concerning availability, standard warranty, and use in critical applications of Texas Instruments semiconductor products and disclaimers thereto appears at the end of this document.

Texas Instruments
108 Wild Basin, Suite 350
Austin, TX 78746
Main: +1-512-279-8800
Fax: +1-512-279-8879
http://www.ti.com/stellaris

Revision Information

This is version 6288 of this document, last updated on August 03, 2010.

Table of Contents

Copyright **2**
Revision Information **2**
1 Introduction **5**
2 Example Applications **7**
2.1 Bit-Banding (bitband) 7
2.2 Blinky (blinky) 7
2.3 Boot Loader Demo 1 (boot_demo1) 7
2.4 Boot Loader Demo 2 (boot_demo2) 8
2.5 Boot Loader (boot_serial) 8
2.6 GPIO JTAG Recovery (gpio_jtag) 8
2.7 Hello World (hello) 8
2.8 Interrupts (interrupts) 8
2.9 MPU (mpu_fault) 9
2.10 PWM (pwmgen) 9
2.11 EK-LM3S811 Quickstart Application (qs_ek-lm3s811) 9
2.12 Timer (timers) 9
2.13 UART (uart_echo) 10
2.14 Watchdog (watchdog) 10
3 Development System Utilities **11**
4 Display Driver **13**
4.1 Introduction 13
4.2 API Functions 13
4.3 Programming Example 16
5 Command Line Processing Module **17**
5.1 Introduction 17
5.2 API Functions 17
5.3 Programming Example 19
6 CPU Usage Module **21**
6.1 Introduction 21
6.2 API Functions 21
6.3 Programming Example 22
7 Flash Parameter Block Module **25**
7.1 Introduction 25
7.2 API Functions 25
7.3 Programming Example 27
8 Integer Square Root Module **29**
8.1 Introduction 29
8.2 API Functions 29
8.3 Programming Example 30
9 Ring Buffer Module **31**
9.1 Introduction 31
9.2 API Functions 31
9.3 Programming Example 37
10 Sine Calculation Module **39**
10.1 Introduction 39
10.2 API Functions 39

10.3 Programming Example 40
11 Software I2C Module 41
11.1 Introduction 41
11.2 API Functions 42
11.3 Programming Example 49
12 Software SSI Module 53
12.1 Introduction 53
12.2 API Functions 54
12.3 Programming Example 66
13 Micro Standard Library Module 71
13.1 Introduction 71
13.2 API Functions 71
13.3 Programming Example 77
14 UART Standard IO Module 79
14.1 Introduction 79
14.2 API Functions 80
14.3 Programming Example 86
IMPORTANT NOTICE 88

1 Introduction

The Texas Instruments® Stellaris® EK-LM3S811 evaluation board is a platform that can be used for software development and to prototype a hardware design. It contains a Stellaris ARM® Cortex™-M3-based microcontroller, along with an OLED display, a push button, and a potentiometer that can be used to exercise the peripherals on the microcontroller. Additionally, all of the microcontroller's pins are brought to unpopulated stake headers, allowing for easy connection to other hardware for the purposes of prototyping (after the stake headers have been populated by the customer).

This document describes the board-specific drivers and example applications that are provided for this development board.

2 Example Applications

The example applications show how to utilize features of the Cortex-M3 microprocessor, the peripherals on the Stellaris microcontroller, and the drivers provided by the peripheral driver library. These applications are intended for demonstration and as a starting point for new applications.

There is an IAR workspace file (`ek-lm3s811.eww`) that contains the peripheral driver library project, along with all of the board example projects, in a single, easy to use workspace for use with Embedded Workbench version 5.

There is a Keil multi-project workspace file (`ek-lm3s811.mpw`) that contains the peripheral driver library project, along with all of the board example projects, in a single, easy to use workspace for use with uVision.

All of these examples reside in the `boards/ek-lm3s811` subdirectory of the firmware development package source distribution.

2.1 Bit-Banding (bitband)

This example application demonstrates the use of the bit-banding capabilities of the Cortex-M3 microprocessor. All of SRAM and all of the peripherals reside within bit-band regions, meaning that bit-banding operations can be applied to any of them. In this example, a variable in SRAM is set to a particular value one bit at a time using bit-banding operations (it would be more efficient to do a single non-bit-banded write; this simply demonstrates the operation of bit-banding).

2.2 Blinky (blinky)

A very simple example that blinks the on-board LED.

2.3 Boot Loader Demo 1 (boot_demo1)

An example to demonstrate the use of the boot loader. After being started by the boot loader, the application will configure the UART and branch back to the boot loader to await the start of an update. The UART will always be configured at 115,200 baud and does not require the use of auto-bauding.

Both the boot loader and the application must be placed into flash. Once the boot loader is in flash, it can be used to program the application into flash as well. Then, the boot loader can be used to replace the application with another.

The boot_demo2 application can be used along with this application to easily demonstrate that the boot loader is actually updating the on-chip flash.

2.4 Boot Loader Demo 2 (boot_demo2)

An example to demonstrate the use of the boot loader. After being started by the boot loader, the application will configure the UART, wait for select button to be pressed, and then branch back to the boot loader to await the start of an update. The UART will always be configured at 115,200 baud and does not require the use of auto-bauding.

Both the boot loader and the application must be placed into flash. Once the boot loader is in flash, it can be used to program the application into flash as well. Then, the boot loader can be used to replace the application with another.

The boot_demo1 application can be used along with this application to easily demonstrate that the boot loader is actually updating the on-chip flash.

2.5 Boot Loader (boot_serial)

The boot loader is a small piece of code that can be programmed at the beginning of flash to act as an application loader as well as an update mechanism for an application running on a Stellaris microcontroller, utilizing either UART0, I2C0, SSI0, or Ethernet. The capabilities of the boot loader are configured via the bl_config.h include file. For this example, the boot loader uses UART0 to load an application.

2.6 GPIO JTAG Recovery (gpio_jtag)

This example demonstrates changing the JTAG pins into GPIOs, along with a mechanism to revert them to JTAG pins. When first run, the pins remain in JTAG mode. Pressing the user push button will toggle the pins between JTAG mode and GPIO mode. Because there is no debouncing of the push button (either in hardware or software), a button press will occasionally result in more than one mode change.

In this example, all five pins (PB7, PC0, PC1, PC2, and PC3) are switched, though the more typical use would be to change PB7 into a GPIO. Note that because of errata in Rev Bx and Rev C0 of Sandstorm-class Stellaris microcontrollers, JTAG and SWD will not function if PB7 is configured as a GPIO. This errata is fixed in Rev C2 of Sandstorm-class Stellaris microcontrollers.

2.7 Hello World (hello)

A very simple “hello world” example. It simply displays “hello world” on the LCD and is a starting point for more complicated applications.

2.8 Interrupts (interrupts)

This example application demonstrates the interrupt preemption and tail-chaining capabilities of Cortex-M3 microprocessor and NVIC. Nested interrupts are synthesized when the interrupts have

the same priority, increasing priorities, and decreasing priorities. With increasing priorities, pre-emption will occur; in the other two cases tail-chaining will occur. The currently pending interrupts and the currently executing interrupt will be displayed on the LCD; GPIO pins D0 through D2 will be asserted upon interrupt handler entry and de-asserted before interrupt handler exit so that the off-to-on time can be observed with a scope or logic analyzer to see the speed of tail-chaining (for the two cases where tail-chaining is occurring).

2.9 MPU (mpu_fault)

This example application demonstrates the use of the MPU to protect a region of memory from access, and to generate a memory management fault when there is an access violation.

2.10 PWM (pwmgen)

This example application utilizes the PWM peripheral to output a 25% duty cycle PWM signal and a 75% duty cycle PWM signal, both at 50 kHz. Once configured, the application enters an infinite loop, doing nothing while the PWM peripheral continues to output its signals.

2.11 EK-LM3S811 Quickstart Application (qs_ek-lm3s811)

A game in which a ship is navigated through an endless tunnel. The potentiometer is used to move the ship up and down, and the user push button is used to fire a missile to destroy obstacles in the tunnel. Score accumulates for survival and for destroying obstacles. The game lasts for only one ship; the score is displayed on the virtual UART at 115,200, 8-N-1 during game play and will be displayed on the screen at the end of the game.

Since the OLED display on the evaluation board has burn-in characteristics similar to a CRT, the application also contains a screen saver. The screen saver will only become active if two minutes have passed without the user push button being pressed while waiting to start the game (that is, it will never come on during game play). An implementation of the Game of Life is run with a field of random data as the seed value.

After two minutes of running the screen saver, the display will be turned off and the user LED will blink. Either mode of screen saver (Game of Life or blank display) will be exited by pressing the user push button. The button will then need to be pressed again to start the game.

2.12 Timer (timers)

This example application demonstrates the use of the timers to generate periodic interrupts. One timer is set up to interrupt once per second and the other to interrupt twice per second; each interrupt handler will toggle its own indicator on the display.

2.13 UART (uart_echo)

This example application utilizes the UART to echo text. The first UART (connected to the FTDI virtual serial port on the Stellaris LM3S811 Evaluation Board) will be configured in 115,200 baud, 8-n-1 mode. All characters received on the UART are transmitted back to the UART.

2.14 Watchdog (watchdog)

This example application demonstrates the use of the watchdog as a simple heartbeat for the system. If the watchdog is not periodically fed, it will reset the system. Each time the watchdog is fed, the LED connected to port C5 is inverted so that it is easy to see that it is being fed, which occurs once every second.

3 Development System Utilities

These are tools that run on the development system, not on the embedded target. They are provided to assist in the development of firmware for Stellaris microcontrollers.

These tools reside in the `tools` subdirectory of the firmware development package source distribution.

Serial Flash Downloader

Usage:

```
sflash [OPTION]... [INPUT FILE]
```

Description:

Downloads a firmware image to a Stellaris board using a UART connection to the Stellaris Serial Flash Loader or the Stellaris Boot Loader. This has the same capabilities as the serial download portion of the Stellaris Flash Programmer.

The source code for this utility is contained in `tools/sflash`, with a pre-built binary contained in `tools/bin`.

Arguments:

-b BAUD specifies the baud rate. If not specified, the default of 115,200 will be used.

-c PORT specifies the COM port. If not specified, the default of COM1 will be used.

-d disables auto-baud.

-h displays usage information.

-l FILENAME specifies the name of the boot loader image file.

-p ADDR specifies the address at which to program the firmware. If not specified, the default of 0 will be used.

-r ADDR specifies the address at which to start processor execution after the firmware has been downloaded. If not specified, the processor will be reset after the firmware has been downloaded.

-s SIZE specifies the size of the data packets used to download the firmware date. This must be a multiple of four between 8 and 252, inclusive. If using the Serial Flash Loader, the maximum value that can be used is 76. If using the Boot Loader, the maximum value that can be used is dependent upon the configuration of the Boot Loader. If not specified, the default of 8 will be used.

INPUT FILE specifies the name of the firmware image file.

Example:

The following will download a firmware image to the board over COM2 without auto-baud support:

```
sflash -c 2 -d image.bin
```

4 Display Driver

Introduction 13
API Functions 13
Programming Example 16

4.1 Introduction

The display driver provides a way to draw text and images on the 96x16 OLED display. The display can also be turned on or off as required in order to preserve the OLED display, which has the same image burn-in characteristics as a CRT display.

This driver supports two displays - the OSRAM unit used in original ek-lm3s811 boards and the newer RIT unit. Determination of which display is in use is made during Display96x16x1Init() and the driver configures for the correct device dynamically. To support only one device and compile out all code and data that is specific for the other, define labels OSRAM_ONLY (to include support for the original, orange OSRAM display) or RIT_ONLY (to support only the blue RIT display) when compiling your application.

This driver is located in `boards/ek-lm3s811/drivers`, with `display96x16x1.c` containing the source code and `display96x16x1.h` containing the API definitions for use by applications.

4.2 API Functions

Functions

- void Display96x16x1Clear (void)
- void Display96x16x1DisplayOff (void)
- void Display96x16x1DisplayOn (void)
- void Display96x16x1ImageDraw (const unsigned char *pucImage, unsigned long ulX, unsigned long ulY, unsigned long ulWidth, unsigned long ulHeight)
- void Display96x16x1Init (tBoolean bFast)
- void Display96x16x1StringDraw (const char *pcStr, unsigned long ulX, unsigned long ulY)

4.2.1 Function Documentation

4.2.1.1 Display96x16x1Clear

Clears the OLED display.

Prototype:

```
void
Display96x16x1Clear(void)
```

Description:

This function will clear the display. All pixels in the display will be turned off.

Returns:
None.

4.2.1.2 Display96x16x1DisplayOff

Turns off the OLED display.

Prototype:
```
void
Display96x16x1DisplayOff(void)
```

Description:
This function will turn off the OLED display. This will stop the scanning of the panel and turn off the on-chip DC-DC converter, preventing damage to the panel due to burn-in (it has similar characters to a CRT in this respect).

Returns:
None.

4.2.1.3 Display96x16x1DisplayOn

Turns on the OLED display.

Prototype:
```
void
Display96x16x1DisplayOn(void)
```

Description:
This function will turn on the OLED display, causing it to display the contents of its internal frame buffer.

Returns:
None.

4.2.1.4 Display96x16x1ImageDraw

Displays an image on the OLED display.

Prototype:
```
void
Display96x16x1ImageDraw(const unsigned char *pucImage,
                        unsigned long ulX,
                        unsigned long ulY,
                        unsigned long ulWidth,
                        unsigned long ulHeight)
```

Parameters:
pucImage is a pointer to the image data.

ulX is the horizontal position to display this image, specified in columns from the left edge of the display.

ulY is the vertical position to display this image, specified in eight scan line blocks from the top of the display (that is, only 0 and 1 are valid).

ulWidth is the width of the image, specified in columns.

ulHeight is the height of the image, specified in eight row blocks (that is, only 1 and 2 are valid).

Description:

This function will display a bitmap graphic on the display. The image to be displayed must be a multiple of eight scan lines high (that is, one row) and will be drawn at a vertical position that is a multiple of eight scan lines (that is, scan line zero or scan line eight, corresponding to row zero or row one).

The image data is organized with the first row of image data appearing left to right, followed immediately by the second row of image data. Each byte contains the data for the eight scan lines of the column, with the top scan line being in the least significant bit of the byte and the bottom scan line being in the most significant bit of the byte.

For example, an image four columns wide and sixteen scan lines tall would be arranged as follows (showing how the eight bytes of the image would appear on the display):

```
+-------+  +-------+  +-------+  +-------+
|   | 0 |  |   | 0 |  |   | 0 |  |   | 0 |
| B | 1 |  | B | 1 |  | B | 1 |  | B | 1 |
| y | 2 |  | y | 2 |  | y | 2 |  | y | 2 |
| t | 3 |  | t | 3 |  | t | 3 |  | t | 3 |
| e | 4 |  | e | 4 |  | e | 4 |  | e | 4 |
|   | 5 |  |   | 5 |  |   | 5 |  |   | 5 |
| 0 | 6 |  | 1 | 6 |  | 2 | 6 |  | 3 | 6 |
|   | 7 |  |   | 7 |  |   | 7 |  |   | 7 |
+-------+  +-------+  +-------+  +-------+

+-------+  +-------+  +-------+  +-------+
|   | 0 |  |   | 0 |  |   | 0 |  |   | 0 |
| B | 1 |  | B | 1 |  | B | 1 |  | B | 1 |
| y | 2 |  | y | 2 |  | y | 2 |  | y | 2 |
| t | 3 |  | t | 3 |  | t | 3 |  | t | 3 |
| e | 4 |  | e | 4 |  | e | 4 |  | e | 4 |
|   | 5 |  |   | 5 |  |   | 5 |  |   | 5 |
| 4 | 6 |  | 5 | 6 |  | 6 | 6 |  | 7 | 6 |
|   | 7 |  |   | 7 |  |   | 7 |  |   | 7 |
+-------+  +-------+  +-------+  +-------+
```

Returns:

None.

4.2.1.5 Display96x16x1Init

Initialize the OLED display.

Prototype:

```
void
Display96x16x1Init(tBoolean bFast)
```

Parameters:

bFast is a boolean that is *true* if the I2C interface should be run at 400 kbps and *false* if it should be run at 100 kbps.

Description:

This function initializes the I2C interface to the OLED display and configures the SSD0303 or SSD1300 controller on the panel.

Returns:

None.

4.2.1.6 Display96x16x1StringDraw

Displays a string on the OLED display.

Prototype:

```
void
Display96x16x1StringDraw(const char *pcStr,
                         unsigned long ulX,
                         unsigned long ulY)
```

Parameters:

pcStr is a pointer to the string to display.

ulX is the horizontal position to display the string, specified in columns from the left edge of the display.

ulY is the vertical position to display the string, specified in eight scan line blocks from the top of the display (that is, only 0 and 1 are valid).

Description:

This function will draw a string on the display. Only the ASCII characters between 32 (space) and 126 (tilde) are supported; other characters will result in random data being draw on the display (based on whatever appears before/after the font in memory). The font is mono-spaced, so characters such as “i” and “l” have more white space around them than characters such as “m” or “w”.

If the drawing of the string reaches the right edge of the display, no more characters will be drawn. Therefore, special care is not required to avoid supplying a string that is “too long” to display.

Returns:

None.

4.3 Programming Example

The following example shows how to use the display driver to display text on the OLED display.

```
//
// Initialize the OLED display using slow I2C mode.
//
Display96x16x1Init(0);

//
// Write text on the display.
//
Display96x16x1StringDraw("Hello", 0, 0);
```

5 Command Line Processing Module

Introduction 17
API Functions 17
Programming Example 19

5.1 Introduction

The command line processor allows a simple command line interface to be made available in an application, for example via a UART. It takes a buffer containing a string (which must be obtained by the application) and breaks it up into a command and arguments (in traditional C “argc, argv” format). The command is then found in a command table and the corresponding function in the table is called to process the command.

This module is contained in `utils/cmdline.c`, with `utils/cmdline.h` containing the API definitions for use by applications.

5.2 API Functions

Data Structures

- tCmdLineEntry

Defines

- CMDLINE_BAD_CMD
- CMDLINE_TOO_MANY_ARGS

Functions

- int CmdLineProcess (char *pcCmdLine)

Variables

- tCmdLineEntry g_sCmdTable[ ]

5.2.1 Data Structure Documentation

5.2.1.1 tCmdLineEntry

Definition:

```
typedef struct
{
    const char *pcCmd;
    pfnCmdLine pfnCmd;
    const char *pcHelp;
}
tCmdLineEntry
```

Members:

pcCmd A pointer to a string containing the name of the command.

pfnCmd A function pointer to the implementation of the command.

pcHelp A pointer to a string of brief help text for the command.

Description:

Structure for an entry in the command list table.

5.2.2 Define Documentation

5.2.2.1 CMDLINE_BAD_CMD

Definition:

```
#define CMDLINE_BAD_CMD
```

Description:

Defines the value that is returned if the command is not found.

5.2.2.2 CMDLINE_TOO_MANY_ARGS

Definition:

```
#define CMDLINE_TOO_MANY_ARGS
```

Description:

Defines the value that is returned if there are too many arguments.

5.2.3 Function Documentation

5.2.3.1 CmdLineProcess

Process a command line string into arguments and execute the command.

Prototype:

```
int
CmdLineProcess(char *pcCmdLine)
```

Parameters:

pcCmdLine points to a string that contains a command line that was obtained by an application by some means.

Description:

This function will take the supplied command line string and break it up into individual arguments. The first argument is treated as a command and is searched for in the command table. If the command is found, then the command function is called and all of the command line arguments are passed in the normal argc, argv form.

The command table is contained in an array named `g_sCmdTable` which must be provided by the application.

Returns:

Returns **CMDLINE_BAD_CMD** if the command is not found, **CMDLINE_TOO_MANY_ARGS** if there are more arguments than can be parsed. Otherwise it returns the code that was returned by the command function.

5.2.4 Variable Documentation

5.2.4.1 g_sCmdTable

Definition:

```
tCmdLineEntry g_sCmdTable[]
```

Description:

This is the command table that must be provided by the application.

5.3 Programming Example

The following example shows how to process a command line.

```
//
// Code for the "foo" command.
//
int
ProcessFoo(int argc, char *argv[])
{
    //
    // Do something, using argc and argv if the command takes arguments.
    //
}

//
// Code for the "bar" command.
//
int
ProcessBar(int argc, char *argv[])
{
    //
    // Do something, using argc and argv if the command takes arguments.
    //
}
```

```
//
// Code for the "help" command.
//
int
ProcessHelp(int argc, char *argv[])
{
    //
    // Provide help.
    //
}

//
// The table of commands supported by this application.
//
tCmdLineEntry g_sCmdTable[] =
{
    { "foo", ProcessFoo, "The first command." },
    { "bar", ProcessBar, "The second command." },
    { "help", ProcessHelp, "Application help." }
};

//
// Read a process a command.
//
int
Test(void)
{
    unsigned char pucCmd[256];

    //
    // Retrieve a command from the user into pucCmd.
    //
    ...

    //
    // Process the command line.
    //
    return(CmdLineProcess(pucCmd));
}
```

6 CPU Usage Module

Introduction 21
API Functions 21
Programming Example 22

6.1 Introduction

The CPU utilization module uses one of the system timers and peripheral clock gating to determine the percentage of the time that the processor is being clocked. For the most part, the processor is executing code whenever it is being clocked (exceptions occur when the clocking is being configured, which only happens at startup, and when entering/exiting an interrupt handler, when the processor is performing stacking operations on behalf of the application).

The specified timer is configured to run when the processor is in run mode and to not run when the processor is in sleep mode. Therefore, the timer will only count when the processor is being clocked. Comparing the number of clocks the timer counted during a fixed period to the number of clocks in the fixed period provides the percentage utilization.

In order for this to be effective, the application must put the processor to sleep when it has no work to do (instead of busy waiting). If the processor never goes to sleep (either because of a continual stream of work to do or a busy loop), the processor utilization will be reported as 100%.

Since deep-sleep mode changes the clocking of the system, the computed processor usage may be incorrect if deep-sleep mode is utilized. The number of clocks the processor spends in run mode will be properly counted, but the timing period may not be accurate (unless extraordinary measures are taken to ensure timing period accuracy).

The accuracy of the computed CPU utilization depends upon the regularity with which CPUUsageTick() is called by the application. If the CPU usage is constant, but CPUUsageTick() is called sporadically, the reported CPU usage will fluctuate as well despite the fact that the CPU usage is actually constant.

This module is contained in `utils/cpu_usage.c`, with `utils/cpu_usage.h` containing the API definitions for use by applications.

6.2 API Functions

Functions

- void CPUUsageInit (unsigned long ulClockRate, unsigned long ulRate, unsigned long ulTimer)
- unsigned long CPUUsageTick (void)

6.2.1 Function Documentation

6.2.1.1 CPUUsageInit

Initializes the CPU usage measurement module.

Prototype:

```
void
CPUUsageInit(unsigned long ulClockRate,
             unsigned long ulRate,
             unsigned long ulTimer)
```

Parameters:

ulClockRate is the rate of the clock supplied to the timer module.

ulRate is the number of times per second that CPUUsageTick() is called.

ulTimer is the index of the timer module to use.

Description:

This function prepares the CPU usage measurement module for measuring the CPU usage of the application.

Returns:

None.

6.2.1.2 CPUUsageTick

Updates the CPU usage for the new timing period.

Prototype:

```
unsigned long
CPUUsageTick(void)
```

Description:

This function, when called at the end of a timing period, will update the CPU usage.

Returns:

Returns the CPU usage percentage as a 16.16 fixed-point value.

6.3 Programming Example

The following example shows how to use the CPU usage module to measure the CPU usage where the foreground simply burns some cycles.

```
//
// The CPU usage for the most recent time period.
//
unsigned long g_ulCPUUsage;

//
// Handles the SysTick interrupt.
```

```
//
void
SysTickIntHandler(void)
{
    //
    // Compute the CPU usage for the last time period.
    //
    g_ulCPUUsage = CPUUsageTick();
}

//
// The main application.
//
int
main(void)
{
    //
    // Initialize the CPU usage module, using timer 0.
    //
    CPUUsageInit(8000000, 100, 0);

    //
    // Initialize SysTick to interrupt at 100 Hz.
    //
    SysTickPeriodSet(8000000 / 100);
    SysTickIntEnable();
    SysTickEnable();

    //
    // Loop forever.
    //
    while(1)
    {
        //
        // Delay for a little bit so that CPU usage is not zero.
        //
        SysCtlDelay(100);

        //
        // Put the processor to sleep.
        //
        SysCtlSleep();
    }
}
```

7 Flash Parameter Block Module

Introduction 25
API Functions 25
Programming Example 27

7.1 Introduction

The flash parameter block module provides a simple, fault-tolerant, persistent storage mechanism for storing parameter information for an application.

The FlashPBInit() function is used to initialize a parameter block. The primary conditions for the parameter block are that flash region used to store the parameter blocks must contain at least two erase blocks of flash to ensure fault tolerance, and the size of the parameter block must be an integral divisor of the the size of an erase block. FlashPBGet() and FlashPBSave() are used to read and write parameter block data into the parameter region. The only constraints on the content of the parameter block are that the first two bytes of the block are reserved for use by the read/write functions as a sequence number and checksum, respectively.

This module is contained in `utils/flash_pb.c`, with `utils/flash_pb.h` containing the API definitions for use by applications.

7.2 API Functions

Functions

- unsigned char * FlashPBGet (void)
- void FlashPBInit (unsigned long ulStart, unsigned long ulEnd, unsigned long ulSize)
- void FlashPBSave (unsigned char *pucBuffer)

7.2.1 Function Documentation

7.2.1.1 FlashPBGet

Gets the address of the most recent parameter block.

Prototype:

```
unsigned char *
FlashPBGet(void)
```

Description:

This function returns the address of the most recent parameter block that is stored in flash.

Returns:

Returns the address of the most recent parameter block, or NULL if there are no valid parameter blocks in flash.

7.2.1.2 FlashPBInit

Initializes the flash parameter block.

Prototype:

```
void
FlashPBInit(unsigned long ulStart,
            unsigned long ulEnd,
            unsigned long ulSize)
```

Parameters:

ulStart is the address of the flash memory to be used for storing flash parameter blocks; this must be the start of an erase block in the flash.

ulEnd is the address of the end of flash memory to be used for storing flash parameter blocks; this must be the start of an erase block in the flash (the first block that is NOT part of the flash memory to be used), or the address of the first word after the flash array if the last block of flash is to be used.

ulSize is the size of the parameter block when stored in flash; this must be a power of two less than or equal to the flash erase block size (typically 1024).

Description:

This function initializes a fault-tolerant, persistent storage mechanism for a parameter block for an application. The last several erase blocks of flash (as specified by *ulStart* and *ulEnd* are used for the storage; more than one erase block is required in order to be fault-tolerant.

A parameter block is an array of bytes that contain the persistent parameters for the application. The only special requirement for the parameter block is that the first byte is a sequence number (explained in FlashPBSave()) and the second byte is a checksum used to validate the correctness of the data (the checksum byte is the byte such that the sum of all bytes in the parameter block is zero).

The portion of flash for parameter block storage is split into N equal-sized regions, where each region is the size of a parameter block (*ulSize*). Each region is scanned to find the most recent valid parameter block. The region that has a valid checksum and has the highest sequence number (with special consideration given to wrapping back to zero) is considered to be the current parameter block.

In order to make this efficient and effective, three conditions must be met. The first is *ulStart* and *ulEnd* must be specified such that at least two erase blocks of flash are dedicated to parameter block storage. If not, fault tolerance can not be guaranteed since an erase of a single block will leave a window where there are no valid parameter blocks in flash. The second condition is that the size (*ulSize*) of the parameter block must be an integral divisor of the size of an erase block of flash. If not, a parameter block will end up spanning between two erase blocks of flash, making it more difficult to manage. The final condition is that the size of the flash dedicated to parameter blocks (*ulEnd* - *ulStart*) divided by the parameter block size (*ulSize*) must be less than or equal to 128. If not, it will not be possible in all cases to determine which parameter block is the most recent (specifically when dealing with the sequence number wrapping back to zero).

When the microcontroller is initially programmed, the flash blocks used for parameter block storage are left in an erased state.

This function must be called before any other flash parameter block functions are called.

Returns:

None.

7.2.1.3 FlashPBSave

Writes a new parameter block to flash.

Prototype:

```
void
FlashPBSave(unsigned char *pucBuffer)
```

Parameters:

pucBuffer is the address of the parameter block to be written to flash.

Description:

This function will write a parameter block to flash. Saving the new parameter blocks involves three steps:

- Setting the sequence number such that it is one greater than the sequence number of the latest parameter block in flash.
- Computing the checksum of the parameter block.
- Writing the parameter block into the storage immediately following the latest parameter block in flash; if that storage is at the start of an erase block, that block is erased first.

By this process, there is always a valid parameter block in flash. If power is lost while writing a new parameter block, the checksum will not match and the partially written parameter block will be ignored. This is what makes this fault-tolerant.

Another benefit of this scheme is that it provides wear leveling on the flash. Since multiple parameter blocks fit into each erase block of flash, and multiple erase blocks are used for parameter block storage, it takes quite a few parameter block saves before flash is re-written.

Returns:

None.

7.3 Programming Example

The following example shows how to use the flash parameter block module to read the contents of a flash parameter block.

```
unsigned char pucBuffer[16], *pucPB;

//
// Initialize the flash parameter block module, using the last two pages of
// a 64 KB device as the parameter block.
//
FlashPBInit(0xf800, 0x10000, 16);

//
// Read the current parameter block.
//
pucPB = FlashPBGet();
if(pucPB)
{
    memcpy(pucBuffer, pucPB);
}
```

8 Integer Square Root Module

Introduction 29
API Functions 29
Programming Example 30

8.1 Introduction

The integer square root module provides an integer version of the square root operation that can be used instead of the floating point version provided in the C library. The algorithm used is a derivative of the manual pencil-and-paper method that used to be taught in school, and is closely related to the pencil-and-paper division method that is likely still taught in school.

For full details of the algorithm, see the article by Jack W. Crenshaw in the February 1998 issue of Embedded System Programming. It can be found online at `http://www.embedded.com/98/9802fe2.htm`.

This module is contained in `utils/isqrt.c`, with `utils/isqrt.h` containing the API definitions for use by applications.

8.2 API Functions

Functions

- unsigned long isqrt (unsigned long ulValue)

8.2.1 Function Documentation

8.2.1.1 isqrt

Compute the integer square root of an integer.

Prototype:

```
unsigned long
isqrt(unsigned long ulValue)
```

Parameters:

ulValue is the value whose square root is desired.

Description:

This function will compute the integer square root of the given input value. Since the value returned is also an integer, it is actually better defined as the largest integer whose square is less than or equal to the input value.

Returns:

Returns the square root of the input value.

8.3 Programming Example

The following example shows how to compute the square root of a number.

```
unsigned long ulValue;

//
// Get the square root of 52378.  The result returned will be 228, which is
// the largest integer less than or equal to the square root of 52378.
//
ulValue = isqrt(52378);
```

9 Ring Buffer Module

Introduction 31
API Functions 31
Programming Example 37

9.1 Introduction

The ring buffer module provides a set of functions allowing management of a block of memory as a ring buffer. This is typically used in buffering transmit or receive data for a communication channel but has many other uses including implementing queues and FIFOs.

This module is contained in `utils/ringbuf.c`, with `utils/ringbuf.h` containing the API definitions for use by applications.

9.2 API Functions

Functions

- void RingBufAdvanceRead (tRingBufObject *ptRingBuf, unsigned long ulNumBytes)
- void RingBufAdvanceWrite (tRingBufObject *ptRingBuf, unsigned long ulNumBytes)
- unsigned long RingBufContigFree (tRingBufObject *ptRingBuf)
- unsigned long RingBufContigUsed (tRingBufObject *ptRingBuf)
- tBoolean RingBufEmpty (tRingBufObject *ptRingBuf)
- void RingBufFlush (tRingBufObject *ptRingBuf)
- unsigned long RingBufFree (tRingBufObject *ptRingBuf)
- tBoolean RingBufFull (tRingBufObject *ptRingBuf)
- void RingBufInit (tRingBufObject *ptRingBuf, unsigned char *pucBuf, unsigned long ulSize)
- void RingBufRead (tRingBufObject *ptRingBuf, unsigned char *pucData, unsigned long ulLength)
- unsigned char RingBufReadOne (tRingBufObject *ptRingBuf)
- unsigned long RingBufSize (tRingBufObject *ptRingBuf)
- unsigned long RingBufUsed (tRingBufObject *ptRingBuf)
- void RingBufWrite (tRingBufObject *ptRingBuf, unsigned char *pucData, unsigned long ulLength)
- void RingBufWriteOne (tRingBufObject *ptRingBuf, unsigned char ucData)

9.2.1 Function Documentation

9.2.1.1 RingBufAdvanceRead

Remove bytes from the ring buffer by advancing the read index.

Prototype:

```
void
RingBufAdvanceRead(tRingBufObject *ptRingBuf,
                   unsigned long ulNumBytes)
```

Parameters:

ptRingBuf points to the ring buffer from which bytes are to be removed.

ulNumBytes is the number of bytes to be removed from the buffer.

Description:

This function advances the ring buffer read index by a given number of bytes, removing that number of bytes of data from the buffer. If *ulNumBytes* is larger than the number of bytes currently in the buffer, the buffer is emptied.

Returns:

None.

9.2.1.2 RingBufAdvanceWrite

Add bytes to the ring buffer by advancing the write index.

Prototype:

```
void
RingBufAdvanceWrite(tRingBufObject *ptRingBuf,
                    unsigned long ulNumBytes)
```

Parameters:

ptRingBuf points to the ring buffer to which bytes have been added.

ulNumBytes is the number of bytes added to the buffer.

Description:

This function should be used by clients who wish to add data to the buffer directly rather than via calls to RingBufWrite() or RingBufWriteOne(). It advances the write index by a given number of bytes. If the *ulNumBytes* parameter is larger than the amount of free space in the buffer, the read pointer will be advanced to cater for the addition. Note that this will result in some of the oldest data in the buffer being discarded.

Returns:

None.

9.2.1.3 RingBufContigFree

Returns number of contiguous free bytes available in a ring buffer.

Prototype:

```
unsigned long
RingBufContigFree(tRingBufObject *ptRingBuf)
```

Parameters:

ptRingBuf is the ring buffer object to check.

Description:
This function returns the number of contiguous free bytes ahead of the current write pointer in the ring buffer.

Returns:
Returns the number of contiguous bytes available in the ring buffer.

9.2.1.4 RingBufContigUsed

Returns number of contiguous bytes of data stored in ring buffer ahead of the current read pointer.

Prototype:
```
unsigned long
RingBufContigUsed(tRingBufObject *ptRingBuf)
```

Parameters:
ptRingBuf is the ring buffer object to check.

Description:
This function returns the number of contiguous bytes of data available in the ring buffer ahead of the current read pointer. This represents the largest block of data which does not straddle the buffer wrap.

Returns:
Returns the number of contiguous bytes available.

9.2.1.5 RingBufEmpty

Determines whether the ring buffer whose pointers and size are provided is empty or not.

Prototype:
```
tBoolean
RingBufEmpty(tRingBufObject *ptRingBuf)
```

Parameters:
ptRingBuf is the ring buffer object to empty.

Description:
This function is used to determine whether or not a given ring buffer is empty. The structure is specifically to ensure that we do not see warnings from the compiler related to the order of volatile accesses being undefined.

Returns:
Returns **true** if the buffer is empty or **false** otherwise.

9.2.1.6 RingBufFlush

Empties the ring buffer.

Prototype:

```
void
RingBufFlush(tRingBufObject *ptRingBuf)
```

Parameters:

ptRingBuf is the ring buffer object to empty.

Description:

Discards all data from the ring buffer.

Returns:

None.

9.2.1.7 RingBufFree

Returns number of bytes available in a ring buffer.

Prototype:

```
unsigned long
RingBufFree(tRingBufObject *ptRingBuf)
```

Parameters:

ptRingBuf is the ring buffer object to check.

Description:

This function returns the number of bytes available in the ring buffer.

Returns:

Returns the number of bytes available in the ring buffer.

9.2.1.8 RingBufFull

Determines whether the ring buffer whose pointers and size are provided is full or not.

Prototype:

```
tBoolean
RingBufFull(tRingBufObject *ptRingBuf)
```

Parameters:

ptRingBuf is the ring buffer object to empty.

Description:

This function is used to determine whether or not a given ring buffer is full. The structure is specifically to ensure that we do not see warnings from the compiler related to the order of volatile accesses being undefined.

Returns:

Returns **true** if the buffer is full or **false** otherwise.

9.2.1.9 RingBufInit

Initialize a ring buffer object.

Prototype:

```
void
RingBufInit(tRingBufObject *ptRingBuf,
            unsigned char *pucBuf,
            unsigned long ulSize)
```

Parameters:

ptRingBuf points to the ring buffer to be initialized.

pucBuf points to the data buffer to be used for the ring buffer.

ulSize is the size of the buffer in bytes.

Description:

This function initializes a ring buffer object, preparing it to store data.

Returns:

None.

9.2.1.10 RingBufRead

Reads data from a ring buffer.

Prototype:

```
void
RingBufRead(tRingBufObject *ptRingBuf,
            unsigned char *pucData,
            unsigned long ulLength)
```

Parameters:

ptRingBuf points to the ring buffer to be read from.

pucData points to where the data should be stored.

ulLength is the number of bytes to be read.

Description:

This function reads a sequence of bytes from a ring buffer.

Returns:

None.

9.2.1.11 RingBufReadOne

Reads a single byte of data from a ring buffer.

Prototype:

```
unsigned char
RingBufReadOne(tRingBufObject *ptRingBuf)
```

Parameters:
ptRingBuf points to the ring buffer to be written to.

Description:
This function reads a single byte of data from a ring buffer.

Returns:
The byte read from the ring buffer.

9.2.1.12 RingBufSize

Return size in bytes of a ring buffer.

Prototype:
```
unsigned long
RingBufSize(tRingBufObject *ptRingBuf)
```

Parameters:
ptRingBuf is the ring buffer object to check.

Description:
This function returns the size of the ring buffer.

Returns:
Returns the size in bytes of the ring buffer.

9.2.1.13 RingBufUsed

Returns number of bytes stored in ring buffer.

Prototype:
```
unsigned long
RingBufUsed(tRingBufObject *ptRingBuf)
```

Parameters:
ptRingBuf is the ring buffer object to check.

Description:
This function returns the number of bytes stored in the ring buffer.

Returns:
Returns the number of bytes stored in the ring buffer.

9.2.1.14 RingBufWrite

Writes data to a ring buffer.

Prototype:

```
void
RingBufWrite(tRingBufObject *ptRingBuf,
             unsigned char *pucData,
             unsigned long ulLength)
```

Parameters:

ptRingBuf points to the ring buffer to be written to.

pucData points to the data to be written.

ulLength is the number of bytes to be written.

Description:

This function write a sequence of bytes into a ring buffer.

Returns:

None.

9.2.1.15 RingBufWriteOne

Writes a single byte of data to a ring buffer.

Prototype:

```
void
RingBufWriteOne(tRingBufObject *ptRingBuf,
                unsigned char ucData)
```

Parameters:

ptRingBuf points to the ring buffer to be written to.

ucData is the byte to be written.

Description:

This function writes a single byte of data into a ring buffer.

Returns:

None.

9.3 Programming Example

The following example shows how to pass data through the ring buffer.

```
char pcBuffer[128], pcData[16];
tRingBufObject sRingBuf;

//
// Initialize the ring buffer.
//
RingBufInit(&sRingBuf, pcBuffer, sizeof(pcBuffer));

//
// Write some data into the ring buffer.
//
RingBufWrite(&sRingBuf, "Hello World", 11);
```

```
//
// Read the data out of the ring buffer.
//
RingBufRead(&sRingBuf, pcData, 11);
```

10 Sine Calculation Module

Introduction 39
API Functions 39
Programming Example 40

10.1 Introduction

This module provides a fixed-point sine function. The input angle is a 0.32 fixed-point value that is the percentage of 360 degrees. This has two benefits; the sine function does not have to handle angles that are outside the range of 0 degrees through 360 degrees (in fact, 360 degrees can not be represented since it would wrap to 0 degrees), and the computation of the angle can be simplified since it does not have to deal with wrapping at values that are not natural for binary arithmetic (such as 360 degrees or 2π radians).

A sine table is used to find the approximate value for a given input angle. The table contains 128 entries that range from 0 degrees through 90 degrees and the symmetry of the sine function is used to determine the value between 90 degrees and 360 degrees. The maximum error caused by this table-based approach is 0.00618, which occurs near 0 and 180 degrees.

This module is contained in `utils/sine.c`, with `utils/sine.h` containing the API definitions for use by applications.

10.2 API Functions

Defines

- cosine(ulAngle)

Functions

- long sine (unsigned long ulAngle)

10.2.1 Define Documentation

10.2.1.1 cosine

Computes an approximation of the cosine of the input angle.

Definition:

```
#define cosine(ulAngle)
```

Parameters:

ulAngle is an angle expressed as a 0.32 fixed-point value that is the percentage of the way around a circle.

Description:

This function computes the cosine for the given input angle. The angle is specified in 0.32 fixed point format, and is therefore always between 0 and 360 degrees, inclusive of 0 and exclusive of 360.

Returns:

Returns the cosine of the angle, in 16.16 fixed point format.

10.2.2 Function Documentation

10.2.2.1 sine

Computes an approximation of the sine of the input angle.

Prototype:

```
long
sine(unsigned long ulAngle)
```

Parameters:

ulAngle is an angle expressed as a 0.32 fixed-point value that is the percentage of the way around a circle.

Description:

This function computes the sine for the given input angle. The angle is specified in 0.32 fixed point format, and is therefore always between 0 and 360 degrees, inclusive of 0 and exclusive of 360.

Returns:

Returns the sine of the angle, in 16.16 fixed point format.

10.3 Programming Example

The following example shows how to produce a sine wave with 7 degrees between successive values.

```
unsigned long ulValue;

//
// Produce a sine wave with each step being 7 degrees advanced from the
// previous.
//
for(ulValue = 0; ; ulValue += 0x04FA4FA4)
{
    //
    // Compute the sine at this angle and do something with the result.
    //
    sine(ulValue);
}
```

11 Software I2C Module

Introduction 41
API Functions 42
Programming Example 49

11.1 Introduction

The software I2C module uses a timer and two GPIO pins to create a slow-speed software I2C peripheral. Multiple software I2C modules can be in use simultaneously, limited only by the availability of resources (RAM for the per-instance data structure, pins for the interface, timers if unique clock rates are required, and processor cycles to execute the code). The software I2C module supports master mode only; multi-master support is not provided. A callback mechanism is used to simulate the interrupts that would be provided by a hardware I2C module.

The API for the software I2C module has been constructed to be as close as possible to the API provided in the Stellaris Peripheral Driver Library for the hardware I2C module. The two notable differences are the function prefix being “SoftI2C” instead of “I2CMaster”, and the first argument of each API is a pointer to the tSoftI2C data structure instead of the base address of the hardware module.

Timing for the software I2C module is provided by the application. The SoftI2CTimerTick() function must be called on a periodic basis to provide the timing for the software I2C module. The timer tick function must be called at four times the desired I2C clock rate; for example, to operate the software I2C interface at 10 KHz, the tick function must be called at a 40 KHz rate. By having the application providing the timing, the timer resource can be flexible and multiple software I2C modules can be driven from a single timer resource. Alternatively, if the software I2C module is only needed for brief periods of time and processor usage is not a concern, the timer tick function can simply be called in a loop until the entire I2C transaction has completed (maximizing both I2C clock speed and processor usage, but not requiring a timer).

The software I2C module requires two GPIO pins; one for SCL and one for SDA. The per-instance data structure is approximately 20 bytes in length (the actual length depends on how the structure is packed by the compiler).

As a point of reference, the following are some rough measurements of the processor usage of the software I2C module at various I2C clock speeds with the processor running at 50 MHz. Actual processor usage may vary, depending on how the application uses the software I2C module, processor clock speed, interrupt priority, and compiler.

I2C Clock	% Of Processor	Million Cycles Per Second
5 KHz	4.53	2.26
10 KHz	9.05	4.52
15 KHz	13.53	6.76
20 KHz	18.03	9.01
25 KHz	22.51	11.25
30 KHz	27.05	13.52
35 KHz	31.52	15.76
40 KHz	36.06	18.03
45 KHz	40.54	20.27
50 KHz	44.96	22.48

This module is contained in `utils/softi2c.c`, with `utils/softi2c.h` containing the API definitions for use by applications.

11.2 API Functions

Data Structures

- tSoftI2C

Functions

- tBoolean SoftI2CBusy (tSoftI2C *pI2C)
- void SoftI2CCallbackSet (tSoftI2C *pI2C, void (*pfnCallback)(void))
- void SoftI2CControl (tSoftI2C *pI2C, unsigned long ulCmd)
- unsigned long SoftI2CDataGet (tSoftI2C *pI2C)
- void SoftI2CDataPut (tSoftI2C *pI2C, unsigned char ucData)
- unsigned long SoftI2CErr (tSoftI2C *pI2C)
- void SoftI2CInit (tSoftI2C *pI2C)
- void SoftI2CIntClear (tSoftI2C *pI2C)
- void SoftI2CIntDisable (tSoftI2C *pI2C)
- void SoftI2CIntEnable (tSoftI2C *pI2C)
- tBoolean SoftI2CIntStatus (tSoftI2C *pI2C, tBoolean bMasked)
- void SoftI2CSCLGPIOSet (tSoftI2C *pI2C, unsigned long ulBase, unsigned char ucPin)
- void SoftI2CSDAGPIOSet (tSoftI2C *pI2C, unsigned long ulBase, unsigned char ucPin)
- void SoftI2CSlaveAddrSet (tSoftI2C *pI2C, unsigned char ucSlaveAddr, tBoolean bReceive)
- void SoftI2CTimerTick (tSoftI2C *pI2C)

11.2.1 Data Structure Documentation

11.2.1.1 tSoftI2C

Definition:

```
typedef struct
{
    void (*pfnIntCallback)(void);
    unsigned long ulSCLGPIO;
    unsigned long ulSDAGPIO;
    unsigned char ucFlags;
    unsigned char ucSlaveAddr;
    unsigned char ucData;
    unsigned char ucState;
    unsigned char ucCurrentBit;
    unsigned char ucIntMask;
    unsigned char ucIntStatus;
}
tSoftI2C
```

Members:

pfnIntCallback The address of the callback function that is called to simulate the interrupts that would be produced by a hardware I2C implementation. This address can be set via a direct structure access or using the SoftI2CCallbackSet function.

ulSCLGPIO The address of the GPIO pin to be used for the SCL signal. This member can be set via a direct structure access or using the SoftI2CSCLGPIOSet function.

ulSDAGPIO The address of the GPIO pin to be used for the SDA signal. This member can be set via a direct structure access or using the SoftI2CSDAGPIOSet function.

ucFlags The flags that control the operation of the SoftI2C module. This member should not be accessed or modified by the application.

ucSlaveAddr The slave address that is currently being accessed. This member should not be accessed or modified by the application.

ucData The data that is currently being transmitted or received. This member should not be accessed or modified by the application.

ucState The current state of the SoftI2C state machine. This member should not be accessed or modified by the application.

ucCurrentBit The number of bits that have been transmitted and received in the current frame. This member should not be accessed or modified by the application.

ucIntMask The set of virtual interrupts that should be sent to the callback function. This member should not be accessed or modified by the application.

ucIntStatus The set of virtual interrupts that are currently asserted. This member should not be accessed or modified by the application.

Description:

This structure contains the state of a single instance of a SoftI2C module.

11.2.2 Function Documentation

11.2.2.1 SoftI2CBusy

Indicates whether or not the SoftI2C module is busy.

Prototype:

```
tBoolean
SoftI2CBusy(tSoftI2C *pI2C)
```

Parameters:

pI2C specifies the SoftI2C data structure.

Description:

This function returns an indication of whether or not the SoftI2C module is busy transmitting or receiving data.

Returns:

Returns **true** if the SoftI2C module is busy; otherwise, returns **false**.

11.2.2.2 SoftI2CCallbackSet

Sets the callback used by the SoftI2C module.

Prototype:

```
void
SoftI2CCallbackSet(tSoftI2C *pI2C,
                   void (*pfnCallback)(void))
```

Parameters:

pI2C specifies the SoftI2C data structure.

pfnCallback is a pointer to the callback function.

Description:

This function sets the address of the callback function that is called when there is an “interrupt” produced by the SoftI2C module.

Returns:

None.

11.2.2.3 SoftI2CControl

Controls the state of the SoftI2C module.

Prototype:

```
void
SoftI2CControl(tSoftI2C *pI2C,
               unsigned long ulCmd)
```

Parameters:

pI2C specifies the SoftI2C data structure.

ulCmd command to be issued to the SoftI2C module.

Description:

This function is used to control the state of the SoftI2C module send and receive operations. The *ucCmd* parameter can be one of the following values:

- **SOFTI2C_CMD_SINGLE_SEND**
- **SOFTI2C_CMD_SINGLE_RECEIVE**
- **SOFTI2C_CMD_BURST_SEND_START**
- **SOFTI2C_CMD_BURST_SEND_CONT**
- **SOFTI2C_CMD_BURST_SEND_FINISH**
- **SOFTI2C_CMD_BURST_SEND_ERROR_STOP**
- **SOFTI2C_CMD_BURST_RECEIVE_START**
- **SOFTI2C_CMD_BURST_RECEIVE_CONT**
- **SOFTI2C_CMD_BURST_RECEIVE_FINISH**
- **SOFTI2C_CMD_BURST_RECEIVE_ERROR_STOP**

Returns:

None.

11.2.2.4 SoftI2CDataGet

Receives a byte that has been sent to the SoftI2C module.

Prototype:

```
unsigned long
SoftI2CDataGet(tSoftI2C *pI2C)
```

Parameters:

pI2C specifies the SoftI2C data structure.

Description:

This function reads a byte of data from the SoftI2C module that was received as a result of an appropriate call to SoftI2CControl().

Returns:

Returns the byte received by the SoftI2C module, cast as an unsigned long.

11.2.2.5 SoftI2CDataPut

Transmits a byte from the SoftI2C module.

Prototype:

```
void
SoftI2CDataPut(tSoftI2C *pI2C,
               unsigned char ucData)
```

Parameters:

pI2C specifies the SoftI2C data structure.

ucData data to be transmitted from the SoftI2C module.

Description:

This function places the supplied data into SoftI2C module in preparation for being transmitted via an appropriate call to SoftI2CControl().

Returns:

None.

11.2.2.6 SoftI2CErr

Gets the error status of the SoftI2C module.

Prototype:

```
unsigned long
SoftI2CErr(tSoftI2C *pI2C)
```

Parameters:

pI2C specifies the SoftI2C data structure.

Description:

This function is used to obtain the error status of the SoftI2C module send and receive operations.

Returns:
Returns the error status, as one of **SOFTI2C_ERR_NONE**, **SOFTI2C_ERR_ADDR_ACK**, or **SOFTI2C_ERR_DATA_ACK**.

11.2.2.7 SoftI2CInit

Initializes the SoftI2C module.

Prototype:
```
void
SoftI2CInit(tSoftI2C *pI2C)
```

Parameters:
pI2C specifies the SoftI2C data structure.

Description:
This function initializes operation of the SoftI2C module. After successful initialization of the SoftI2C module, the software I2C bus is in the idle state.

Returns:
None.

11.2.2.8 SoftI2CIntClear

Clears the SoftI2C “interrupt”.

Prototype:
```
void
SoftI2CIntClear(tSoftI2C *pI2C)
```

Parameters:
pI2C specifies the SoftI2C data structure.

Description:
The SoftI2C “interrupt” source is cleared, so that it no longer asserts. This function must be called in the “interrupt” handler to keep it from being called again immediately on exit.

Returns:
None.

11.2.2.9 SoftI2CIntDisable

Disables the SoftI2C “interrupt”.

Prototype:
```
void
SoftI2CIntDisable(tSoftI2C *pI2C)
```

Parameters:
pI2C specifies the SoftI2C data structure.

Description:
Disables the SoftI2C “interrupt” source.

Returns:
None.

11.2.2.10 SoftI2CIntEnable

Enables the SoftI2C “interrupt”.

Prototype:
```
void
SoftI2CIntEnable(tSoftI2C *pI2C)
```

Parameters:
pI2C specifies the SoftI2C data structure.

Description:
Enables the SoftI2C “interrupt” source.

Returns:
None.

11.2.2.11 SoftI2CIntStatus

Gets the current SoftI2C “interrupt” status.

Prototype:
```
tBoolean
SoftI2CIntStatus(tSoftI2C *pI2C,
                 tBoolean bMasked)
```

Parameters:
pI2C specifies the SoftI2C data structure.
bMasked is **false** if the raw “interrupt” status is requested and **true** if the masked “interrupt” status is requested.

Description:
This returns the “interrupt” status for the SoftI2C module. Either the raw “interrupt” status or the status of “interrupts” that are allowed to reflect to the processor can be returned.

Returns:
The current interrupt status, returned as **true** if active or **false** if not active.

11.2.2.12 SoftI2CSCLGPIOSet

Sets the GPIO pin to be used as the SoftI2C SCL signal.

Prototype:

```
void
SoftI2CSCLGPIOSet(tSoftI2C *pI2C,
                  unsigned long ulBase,
                  unsigned char ucPin)
```

Parameters:

pI2C specifies the SoftI2C data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftI2C SCL signal.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

11.2.2.13 SoftI2CSDAGPIOSet

Sets the GPIO pin to be used as the SoftI2C SDA signal.

Prototype:

```
void
SoftI2CSDAGPIOSet(tSoftI2C *pI2C,
                  unsigned long ulBase,
                  unsigned char ucPin)
```

Parameters:

pI2C specifies the SoftI2C data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftI2C SDA signal.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

11.2.2.14 SoftI2CSlaveAddrSet

Sets the address that the SoftI2C module places on the bus.

Prototype:

```
void
SoftI2CSlaveAddrSet(tSoftI2C *pI2C,
```

```
                        unsigned char ucSlaveAddr,
                        tBoolean bReceive)
```

Parameters:

pI2C specifies the SoftI2C data structure.

ucSlaveAddr 7-bit slave address

bReceive flag indicating the type of communication with the slave.

Description:

This function sets the address that the SoftI2C module places on the bus when initiating a transaction. When the *bReceive* parameter is set to **true**, the address indicates that the SoftI2C moudle is initiating a read from the slave; otherwise the address indicates that the SoftI2C module is initiating a write to the slave.

Returns:

None.

11.2.2.15 SoftI2CTimerTick

Performs the periodic update of the SoftI2C module.

Prototype:

```
void
SoftI2CTimerTick(tSoftI2C *pI2C)
```

Parameters:

pI2C specifies the SoftI2C data structure.

Description:

This function performs the periodic, time-based updates to the SoftI2C module. The transmission and reception of data over the SoftI2C link is performed by the state machine in this function.

This function must be called at four times the desired SoftI2C clock rate. For example, to run the SoftI2C clock at 10 KHz, this function must be called at a 40 KHz rate.

Returns:

None.

11.3 Programming Example

The following example shows how to configure the software I2C module and transmit some data to an external peripheral. This example uses Timer 0 as the timing source.

```
//
// The instance data for the software I2C.
//
tSoftI2C g_sI2C;

//
// The timer tick function.
```

```
//
void
Timer0AIntHandler(void)
{
    //
    // Clear the timer interrupt.
    //
    TimerIntClear(TIMER0_BASE, TIMER_TIMA_TIMEOUT);

    //
    // Call the software I2C timer tick function.
    //
    SoftI2CTimerTick(&g_sI2C);
}

//
// The callback function for the software I2C.  This function is equivalent
// to the interrupt handler for a hardware I2C.
//
void
I2CCallback(void)
{
    //
    // Clear the interrupt.
    //
    SoftI2CIntClear(&g_sI2C);

    //
    // Handle the interrupt.
    //
    ...
}

//
// Setup the software I2C and send some data.
//
void
TestSoftI2C(void)
{
    //
    // Clear the software I2C instance data.
    //
    memset(&g_sI2C, 0, sizeof(g_sI2C));

    //
    // Set the callback function used for this software I2C.
    //
    SoftI2CCallbackSet(&g_sI2C, I2CCallback);

    //
    // Configure the pins used for the software I2C.  This example uses
    // pins PD0 and PE1.
    //
    SoftI2CSCLGPIOSet(&g_sI2C, GPIO_PORTD_BASE, GPIO_PIN_0);
    SoftI2CSDAGPIOSet(&g_sI2C, GPIO_PORTE_BASE, GPIO_PIN_1);

    //
    // Enable the GPIO modules that contains the GPIO pins to be used by
    // the software I2C.
    //
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOD);
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);

    //
    // Initalize the software I2C module.
    //
```

```
    SoftI2CInit(&g_sI2C);

    //
    // Configure the timer used to generate the timing for the software
    // I2C.  The interface will be run at 10 KHz, requiring a timer tick
    // at 40 KHz.
    //
    SysCtlPeripheralEnable(SYSCTL_PERIPH_TIMER0);
    TimerConfigure(TIMER0_BASE, TIMER_CFG_32_BIT_PER);
    TimerLoadSet(TIMER0_BASE, TIMER_A, SysCtlClockGet() / 40000);
    TimerIntEnable(TIMER0_BASE, TIMER_TIMA_TIMEOUT);
    IntEnable(INT_TIMER0A);
    TimerEnable(TIMER0_BASE, TIMER_A);

    //
    // Enable the software I2C interrupt.
    //
    SoftI2CIntEnable(&g_sI2C);

    //
    // Send a single byte to the slave device.
    //
    SoftI2CSlaveAddrSet(&g_sI2C, 0x55, 0);
    SoftI2CDataPut(&g_sI2C, 0xaa);
    SoftI2CControl(&g_sI2C, SOFTI2C_CMD_SINGLE_SEND);

    //
    // Wait until the software I2C is idle.  The completion interrupt will
    // be sent to the callback function prior to exiting this loop.
    //
    while(SoftI2CBusy(&g_sI2C))
    {
    }
}
```

As a comparison, the following is the equivalent code using the hardware I2C module and the Stellaris Peripheral Driver Library.

```
//
// The interrupt handler for the hardware I2C.
//
void
I2C0IntHandler(void)
{
    //
    // Clear the asserted interrupt sources.
    //
    I2CMasterIntClear(I2C0_MASTER_BASE);

    //
    // Handle the interrupt.
    //
    ...
}

//
// Setup the hardware I2C and send some data.
//
void
TestI2C(void)
{
    //
    // Enable the GPIO module that contains the GPIO pins to be used by
    // the I2C, as well as the I2C module.
    //
```

```
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOB);
    SysCtlPeripheralEnable(SYSCTL_PERIPH_I2C0);

    //
    // Configure the GPIO pins for use by the I2C module.
    //
    GPIOPinTypeI2C(GPIO_PORTB_BASE, GPIO_PIN_2 | GPIO_PIN_3);

    //
    // Initalize the hardware I2C module.
    //
    I2CMasterInitExpClk(I2C0_MASTER_BASE, SysCtlClockGet(), false);

    //
    // Enable the hardware I2C.
    //
    I2CMasterEnable(I2C0_MASTER_BASE);

    //
    // Enable the interrupt in the hardware I2C.
    //
    I2CMasterIntEnable(I2C0_MASTER_BASE);
    IntEnable(INT_I2C0);

    //
    // Write some data into the hardware I2C transmit FIFO.
    //
    I2CMasterSlaveAddrSet(I2C0_MASTER_BASE, 0x55, 0);
    I2CMasterDataPut(I2C0_MASTER_BASE, 0xaa);
    I2CMasterControl(I2C0_MASTER_BASE, I2C_MASTER_CMD_SINGLE_SEND);

    //
    // Wait until the hardware I2C is idle.  The interrupt will be sent to
    // the interrupt handler prior to exiting this loop.
    //
    while(I2CBusy(I2C0_MASTER_BASE))
    {
    }
}
```

12 Software SSI Module

Introduction 53
API Functions 54
Programming Example 66

12.1 Introduction

The software SSI module uses a timer and a few GPIO pins to create a slow-speed software SSI peripheral. Multiple software SSI modules can be in use simultaneously, limited only by the availability of resources (RAM for the per-instance data structure, pins for the interface, timers if unique clock rates are required, and processor cycles to execute the code). The software SSI module supports the Motorola® SPI™ formats with 4 to 16 data bits. A callback mechanism is used to simulate the interrupts that would be provided by a hardware SSI module.

The API for the software SSI module has been constructed to be as close as possible to the API provided in the Stellaris Peripheral Driver Library for the hardware SSI module. The two notable difference are the function prefix being “SoftSSI” instead of “SSI”, and the first argument of each API is a pointer to the tSoftSSI data structure instead of the base address of the hardware module.

Timing for the software SSI module is provided by the application. The SoftSSITimerTick() function must be called on a periodic basis to provide the timing for the software SSI module. The timer tick function must be called at twice the desired SSI clock rate; for example, to operate the software SSI interface at 10 KHz, the tick function must be called at a 20 KHz rate. By having the application providing the timing, the timer resource to be used is flexible and multiple software SSI modules can be driven from a single timer resource. Alternatively, if the software SSI module is only needed for brief periods of time and processor usage is not a concern, the timer tick function can simply be called in a loop until the entire SSI transaction has completed (maximizing both SSI clock speed and processor usage, but not requiring a timer).

The software SSI module requires a few as two and as many as four GPIO pins. The following table shows the possible pin usages for the software SSI module:

Fss	Clk	Tx	Rx	Pins	Description
	yes	yes		2	transmit only
yes	yes	yes		3	
	yes		yes	2	receive only
yes	yes		yes	3	
	yes	yes	yes	3	transmit and receive
yes	yes	yes	yes	4	

For the cases where Fss is not used, it is up to the application to control that signal (either via a separately-controlled GPIO, or by being tied to ground in the hardware).

The per-instance data structure is approximately 52 bytes in length (the actual length will depend upon how the structure is packed by the compiler in use).

As a point of reference, the following are some rough measurements of the processor usage of the software SSI module at various SSI clock speeds with the processor running at 50 MHz. Actual processor usage may vary, depending upon how the application uses the software SSI module, processor clock speed, interrupt priority, and compiler in use.

SSI Clock	% Of Processor	Million Cycles Per Second
10 KHz	5.26	2.63
20 KHz	10.48	5.24
30 KHz	15.68	7.84
40 KHz	20.90	10.45
50 KHz	26.10	13.05
60 KHz	31.38	15.69
70 KHz	36.54	18.27
80 KHz	41.79	20.89
90 KHz	47.06	23.53
100 KHz	52.17	26.08

This module is contained in `utils/softssi.c`, with `utils/softssi.h` containing the API definitions for use by applications.

12.2 API Functions

Data Structures

- tSoftSSI

Functions

- tBoolean SoftSSIBusy (tSoftSSI *pSSI)
- void SoftSSICallbackSet (tSoftSSI *pSSI, void (*pfnCallback)(void))
- void SoftSSIClkGPIOSet (tSoftSSI *pSSI, unsigned long ulBase, unsigned char ucPin)
- void SoftSSIConfigSet (tSoftSSI *pSSI, unsigned char ucProtocol, unsigned char ucBits)
- tBoolean SoftSSIDataAvail (tSoftSSI *pSSI)
- void SoftSSIDataGet (tSoftSSI *pSSI, unsigned long *pulData)
- long SoftSSIDataGetNonBlocking (tSoftSSI *pSSI, unsigned long *pulData)
- void SoftSSIDataPut (tSoftSSI *pSSI, unsigned long ulData)
- long SoftSSIDataPutNonBlocking (tSoftSSI *pSSI, unsigned long ulData)
- void SoftSSIDisable (tSoftSSI *pSSI)
- void SoftSSIEnable (tSoftSSI *pSSI)
- void SoftSSIFssGPIOSet (tSoftSSI *pSSI, unsigned long ulBase, unsigned char ucPin)
- void SoftSSIIntClear (tSoftSSI *pSSI, unsigned long ulIntFlags)
- void SoftSSIIntDisable (tSoftSSI *pSSI, unsigned long ulIntFlags)
- void SoftSSIIntEnable (tSoftSSI *pSSI, unsigned long ulIntFlags)
- unsigned long SoftSSIIntStatus (tSoftSSI *pSSI, tBoolean bMasked)
- void SoftSSIRxBufferSet (tSoftSSI *pSSI, unsigned short *pusRxBuffer, unsigned short usLen)
- void SoftSSIRxGPIOSet (tSoftSSI *pSSI, unsigned long ulBase, unsigned char ucPin)
- tBoolean SoftSSISpaceAvail (tSoftSSI *pSSI)
- void SoftSSITimerTick (tSoftSSI *pSSI)
- void SoftSSITxBufferSet (tSoftSSI *pSSI, unsigned short *pusTxBuffer, unsigned short usLen)
- void SoftSSITxGPIOSet (tSoftSSI *pSSI, unsigned long ulBase, unsigned char ucPin)

12.2.1 Data Structure Documentation

12.2.1.1 tSoftSSI

Definition:

```
typedef struct
{
    void (*pfnIntCallback)(void);
    unsigned long ulFssGPIO;
    unsigned long ulClkGPIO;
    unsigned long ulTxGPIO;
    unsigned long ulRxGPIO;
    unsigned short *pusTxBuffer;
    unsigned short *pusRxBuffer;
    unsigned short usTxBufferLen;
    unsigned short usTxBufferRead;
    unsigned short usTxBufferWrite;
    unsigned short usRxBufferLen;
    unsigned short usRxBufferRead;
    unsigned short usRxBufferWrite;
    unsigned short usTxData;
    unsigned short usRxData;
    unsigned char ucFlags;
    unsigned char ucBits;
    unsigned char ucState;
    unsigned char ucCurrentBit;
    unsigned char ucIntMask;
    unsigned char ucIntStatus;
    unsigned char ucIdleCount;
}
tSoftSSI
```

Members:

pfnIntCallback The address of the callback function that is called to simulate the interrupts that would be produced by a hardware SSI implementation. This address can be set via a direct structure access or using the SoftSSICallbackSet function.

ulFssGPIO The address of the GPIO pin to be used for the Fss signal. If this member is zero, the Fss signal is not generated. This member can be set via a direct structure access or using the SoftSSIFssGPIOSet function.

ulClkGPIO The address of the GPIO pin to be used for the Clk signal. This member can be set via a direct structure access or using the SoftSSIClkGPIOSet function.

ulTxGPIO The address of the GPIO pin to be used for the Tx signal. This member can be set via a direct structure access or using the SoftSSITxGPIOSet function.

ulRxGPIO The address of the GPIO pin to be used for the Rx signal. If this member is zero, the Rx signal is not read. This member can be set via a direct structure access or using the SoftSSIRxGPIOSet function.

pusTxBuffer The address of the data buffer used for the transmit FIFO. This member can be set via a direct structure access or using the SoftSSITxBufferSet function.

pusRxBuffer The address of the data buffer used for the receive FIFO. This member can be set via a direct structure access or using the SoftSSIRxBufferSet function.

usTxBufferLen The length of the transmit FIFO. This member can be set via a direct structure access or using the SoftSSITxBufferSet function.

usTxBufferRead The index into the transmit FIFO of the next word to be transmitted. This member should be initialized to zero, but should not be accessed or modified by the application.

usTxBufferWrite The index into the transmit FIFO of the next location to store data into the FIFO. This member should be initialized to zero, but should not be accessed or modified by the application.

usRxBufferLen The length of the receive FIFO. This member can be set via a direct structure access or using the SoftSSIRxBufferSet function.

usRxBufferRead The index into the receive FIFO of the next word to be read from the FIFO. This member should be initialized to zero, but should not be accessed or modified by the application.

usRxBufferWrite The index into the receive FIFO of the location to store the next word received. This member should be initialized to zero, but should not be accessed or modified by the application.

usTxData The word that is currently being transmitted. This member should not be accessed or modified by the application.

usRxData The word that is currently being received. This member should not be accessed or modified by the application.

ucFlags The flags that control the operation of the SoftSSI module. This member should not be accessed or modified by the application.

ucBits The number of data bits in each SoftSSI frame, which also specifies the width of each data item in the transmit and receive FIFOs. This member can be set via a direct structure access or using the SoftSSIConfigSet function.

ucState The current state of the SoftSSI state machine. This member should not be accessed or modified by the application.

ucCurrentBit The number of bits that have been transmitted and received in the current frame. This member should not be accessed or modified by the application.

ucIntMask The set of virtual interrupts that should be sent to the callback function. This member should not be accessed or modified by the application.

ucIntStatus The set of virtual interrupts that are currently asserted. This member should not be accessed or modified by the application.

ucIdleCount The number of tick counts that the SoftSSI module has been idle with data stored in the receive FIFO, which is used to generate the receive timeout interrupt. This member should not be accessed or modified by the application.

Description:

This structure contains the state of a single instance of a SoftSSI module.

12.2.2 Function Documentation

12.2.2.1 SoftSSIBusy

Determines whether the SoftSSI transmitter is busy or not.

Prototype:

```
tBoolean
SoftSSIBusy(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

Allows the caller to determine whether all transmitted bytes have cleared the transmitter. If **false** is returned, then the transmit FIFO is empty and all bits of the last transmitted word have left the shift register.

Returns:

Returns **true** if the SoftSSI is transmitting or **false** if all transmissions are complete.

12.2.2.2 SoftSSICallbackSet

Sets the callback used by the SoftSSI module.

Prototype:

```
void
SoftSSICallbackSet(tSoftSSI *pSSI,
                   void (*pfnCallback)(void))
```

Parameters:

pSSI specifies the SoftSSI data structure.

pfnCallback is a pointer to the callback function.

Description:

This function sets the address of the callback function that is called when there is an “interrupt” produced by the SoftSSI module.

Returns:

None.

12.2.2.3 SoftSSIClkGPIOSet

Sets the GPIO pin to be used as the SoftSSI Clk signal.

Prototype:

```
void
SoftSSIClkGPIOSet(tSoftSSI *pSSI,
                  unsigned long ulBase,
                  unsigned char ucPin)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftSSI Clk signal.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

12.2.2.4 SoftSSIConfigSet

Sets the configuration of a SoftSSI module.

Prototype:

```
void
SoftSSIConfigSet(tSoftSSI *pSSI,
                 unsigned char ucProtocol,
                 unsigned char ucBits)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ucProtocol specifes the data transfer protocol.

ucBits specifies the number of bits transferred per frame.

Description:

This function configures the data format of a SoftSSI module. The *ucProtocol* parameter can be one of the following values: **SOFTSSI_FRF_MOTO_MODE_0**, **SOFTSSI_FRF_MOTO_MODE_1**, **SOFTSSI_FRF_MOTO_MODE_2**, or **SOFTSSI_FRF_MOTO_MODE_3**. These frame formats imply the following polarity and phase configurations:

```
Polarity Phase         Mode
  0       0    SOFTSSI_FRF_MOTO_MODE_0
  0       1    SOFTSSI_FRF_MOTO_MODE_1
  1       0    SOFTSSI_FRF_MOTO_MODE_2
  1       1    SOFTSSI_FRF_MOTO_MODE_3
```

The *ucBits* parameter defines the width of the data transfers, and can be a value between 4 and 16, inclusive.

Returns:

None.

12.2.2.5 SoftSSIDataAvail

Determines if there is any data in the receive FIFO.

Prototype:

```
tBoolean
SoftSSIDataAvail(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

This function determines if there is any data available to be read from the receive FIFO.

Returns:

Returns **true** if there is data in the receive FIFO or **false** if there is no data in the receive FIFO.

12.2.2.6 SoftSSIDataGet

Gets a data element from the SoftSSI receive FIFO.

Prototype:

```
void
SoftSSIDataGet(tSoftSSI *pSSI,
               unsigned long *pulData)
```

Parameters:

pSSI specifies the SoftSSI data structure.

pulData is a pointer to a storage location for data that was received over the SoftSSI interface.

Description:

This function gets received data from the receive FIFO of the specified SoftSSI module and places that data into the location specified by the *pulData* parameter.

Note:

Only the lower N bits of the value written to *pulData* contain valid data, where N is the data width as configured by SoftSSIConfigSet(). For example, if the interface is configured for 8-bit data width, only the lower 8 bits of the value written to *pulData* contain valid data.

Returns:

None.

12.2.2.7 SoftSSIDataGetNonBlocking

Gets a data element from the SoftSSI receive FIFO.

Prototype:

```
long
SoftSSIDataGetNonBlocking(tSoftSSI *pSSI,
                          unsigned long *pulData)
```

Parameters:

pSSI specifies the SoftSSI data structure.

pulData is a pointer to a storage location for data that was received over the SoftSSI interface.

Description:

This function gets received data from the receive FIFO of the specified SoftSSI module and places that data into the location specified by the *ulData* parameter. If there is no data in the FIFO, then this function returns a zero.

Note:

Only the lower N bits of the value written to *pulData* contain valid data, where N is the data width as configured by SoftSSIConfigSet(). For example, if the interface is configured for 8-bit data width, only the lower 8 bits of the value written to *pulData* contain valid data.

Returns:

Returns the number of elements read from the SoftSSI receive FIFO.

12.2.2.8 SoftSSIDataPut

Puts a data element into the SoftSSI transmit FIFO.

Prototype:

```
void
SoftSSIDataPut(tSoftSSI *pSSI,
               unsigned long ulData)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulData is the data to be transmitted over the SoftSSI interface.

Description:

This function places the supplied data into the transmit FIFO of the specified SoftSSI module.

Note:

The upper 32 - N bits of the *ulData* are discarded, where N is the data width as configured by SoftSSIConfigSet(). For example, if the interface is configured for 8-bit data width, the upper 24 bits of *ulData* are discarded.

Returns:

None.

12.2.2.9 SoftSSIDataPutNonBlocking

Puts a data element into the SoftSSI transmit FIFO.

Prototype:

```
long
SoftSSIDataPutNonBlocking(tSoftSSI *pSSI,
                          unsigned long ulData)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulData is the data to be transmitted over the SoftSSI interface.

Description:

This function places the supplied data into the transmit FIFO of the specified SoftSSI module. If there is no space in the FIFO, then this function returns a zero.

Note:

The upper 32 - N bits of the *ulData* are discarded, where N is the data width as configured by SoftSSIConfigSet(). For example, if the interface is configured for 8-bit data width, the upper 24 bits of *ulData* are discarded.

Returns:

Returns the number of elements written to the SSI transmit FIFO.

12.2.2.10 SoftSSIDisable

Disables the SoftSSI module.

Prototype:

```
void
SoftSSIDisable(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

This function disables operation of the SoftSSI module. If a data transfer is in progress, it is finished before the module is fully disabled.

Returns:

None.

12.2.2.11 SoftSSIEnable

Enables the SoftSSI module.

Prototype:

```
void
SoftSSIEnable(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

This function enables operation of the SoftSSI module. The SoftSSI module must be configured before it is enabled.

Returns:

None.

12.2.2.12 SoftSSIFssGPIOSet

Sets the GPIO pin to be used as the SoftSSI Fss signal.

Prototype:

```
void
SoftSSIFssGPIOSet(tSoftSSI *pSSI,
                  unsigned long ulBase,
                  unsigned char ucPin)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftSSI Fss signal. If there is not a GPIO pin allocated for Fss, the SoftSSI module does not assert/deassert the Fss signal, leaving it to the application either to do manually or to not do at all if the slave device has Fss tied to ground.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

12.2.2.13 SoftSSIIntClear

Clears SoftSSI "interrupt" sources.

Prototype:

```
void
SoftSSIIntClear(tSoftSSI *pSSI,
                unsigned long ulIntFlags)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulIntFlags is a bit mask of the "interrupt" sources to be cleared.

Description:

The specified SoftSSI "interrupt" sources are cleared so that they no longer assert. This function must be called in the "interrupt" handler to keep the "interrupt" from being recognized again immediately upon exit. The *ulIntFlags* parameter is the logical OR of any of the **SOFTSSI_TXEOT**, **SOFTSSI_RXTO**, and **SOFTSSI_RXOR** values.

Returns:

None.

12.2.2.14 SoftSSIIntDisable

Disables individual SoftSSI "interrupt" sources.

Prototype:

```
void
SoftSSIIntDisable(tSoftSSI *pSSI,
                  unsigned long ulIntFlags)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulIntFlags is a bit mask of the "interrupt" sources to be disabled.

Description:

Disables the indicated SoftSSI "interrupt" sources. The *ulIntFlags* parameter can be any of the **SOFTSSI_TXEOT**, **SOFTSSI_TXFF**, **SOFTSSI_RXFF**, **SOFTSSI_RXTO**, or **SOFTSSI_RXOR** values.

Returns:
None.

12.2.2.15 SoftSSIIntEnable

Enables individual SoftSSI "interrupt" sources.

Prototype:
```
void
SoftSSIIntEnable(tSoftSSI *pSSI,
                 unsigned long ulIntFlags)
```

Parameters:
pSSI specifies the SoftSSI data structure.
ulIntFlags is a bit mask of the "interrupt" sources to be enabled.

Description:
Enables the indicated SoftSSI "interrupt" sources. Only the sources that are enabled can be reflected to the callback function; disabled sources do not result in a callback. The *ulIntFlags* parameter can be any of the **SOFTSSI_TXEOT**, **SOFTSSI_TXFF**, **SOFTSSI_RXFF**, **SOFTSSI_RXTO**, or **SOFTSSI_RXOR** values.

Returns:
None.

12.2.2.16 SoftSSIIntStatus

Gets the current "interrupt" status.

Prototype:
```
unsigned long
SoftSSIIntStatus(tSoftSSI *pSSI,
                 tBoolean bMasked)
```

Parameters:
pSSI specifies the SoftSSI data structure.
bMasked is **false** if the raw "interrupt" status is required or **true** if the masked "interrupt" status is required.

Description:
This function returns the "interrupt" status for the SoftSSI module. Either the raw "interrupt" status or the status of "interrupts" that are allowed to reflect to the callback can be returned.

Returns:
The current "interrupt" status, enumerated as a bit field of **SOFTSSI_TXEOT**, **SOFTSSI_TXFF**, **SOFTSSI_RXFF**, **SOFTSSI_RXTO**, and **SOFTSSI_RXOR**.

12.2.2.17 SoftSSIRxBufferSet

Sets the receive FIFO buffer for a SoftSSI module.

Prototype:

```
void
SoftSSIRxBufferSet(tSoftSSI *pSSI,
                   unsigned short *pusRxBuffer,
                   unsigned short usLen)
```

Parameters:

pSSI specifies the SoftSSI data structure.

pusRxBuffer is the address of the receive FIFO buffer.

usLen is the size, in 16-bit half-words, of the receive FIFO buffer.

Description:

This function sets the address and size of the receive FIFO buffer and also resets the read and write pointers, marking the receive FIFO as empty. When the buffer pointer and length are configured as zero, all data received from the slave device is discarded. This capability is useful when there is no GPIO pin allocated for the Rx signal.

Returns:

None.

12.2.2.18 SoftSSIRxGPIOSet

Sets the GPIO pin to be used as the SoftSSI Rx signal.

Prototype:

```
void
SoftSSIRxGPIOSet(tSoftSSI *pSSI,
                 unsigned long ulBase,
                 unsigned char ucPin)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftSSI Rx signal. If there is not a GPIO pin allocated for Rx, the SoftSSI module does not read data from the slave device.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

12.2.2.19 SoftSSISpaceAvail

Determines if there is any space in the transmit FIFO.

Prototype:

```
tBoolean
SoftSSISpaceAvail(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

This function determines if there is space available in the transmit FIFO.

Returns:

Returns **true** if there is space available in the transmit FIFO or **false** if there is no space available in the transmit FIFO.

12.2.2.20 SoftSSITimerTick

Performs the periodic update of the SoftSSI module.

Prototype:

```
void
SoftSSITimerTick(tSoftSSI *pSSI)
```

Parameters:

pSSI specifies the SoftSSI data structure.

Description:

This function performs the periodic, time-based updates to the SoftSSI module. The transmission and reception of data over the SoftSSI link is performed by the state machine in this function.

This function must be called at twice the desired SoftSSI clock rate. For example, to run the SoftSSI clock at 10 KHz, this function must be called at a 20 KHz rate.

Returns:

None.

12.2.2.21 SoftSSITxBufferSet

Sets the transmit FIFO buffer for a SoftSSI module.

Prototype:

```
void
SoftSSITxBufferSet(tSoftSSI *pSSI,
                   unsigned short *pusTxBuffer,
                   unsigned short usLen)
```

Parameters:

pSSI specifies the SoftSSI data structure.

pusTxBuffer is the address of the transmit FIFO buffer.

usLen is the size, in 16-bit half-words, of the transmit FIFO buffer.

Description:

This function sets the address and size of the transmit FIFO buffer and also resets the read and write pointers, marking the transmit FIFO as empty.

Returns:

None.

12.2.2.22 SoftSSITxGPIOSet

Sets the GPIO pin to be used as the SoftSSI Tx signal.

Prototype:

```
void
SoftSSITxGPIOSet(tSoftSSI *pSSI,
                 unsigned long ulBase,
                 unsigned char ucPin)
```

Parameters:

pSSI specifies the SoftSSI data structure.

ulBase is the base address of the GPIO module.

ucPin is the bit-packed representation of the pin to use.

Description:

This function sets the GPIO pin that is used for the SoftSSI Tx signal.

The pin is specified using a bit-packed byte, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

12.3 Programming Example

The following example shows how to configure the software SSI module and transmit some data to an external peripheral. This example uses Timer 0 as the timing source.

```
//
// The instance data for the software SSI.
//
tSoftSSI g_sSSI;

//
// The buffer used to hold the transmit data.
//
unsigned short g_pusTxBuffer[8];

//
// The timer tick function.
//
```

```
void
Timer0AIntHandler(void)
{
    //
    // Clear the timer interrupt.
    //
    TimerIntClear(TIMER0_BASE, TIMER_TIMA_TIMEOUT);

    //
    // Call the software SSI timer tick function.
    //
    SoftSSITimerTick(&g_sSSI);
}

//
// The callback function for the software SSI.  This function is equivalent
// to the interrupt handler for a hardware SSI.
//
void
SSICallback(void)
{
    unsigned long ulInts;

    //
    // Read the asserted interrupt sources.
    //
    ulInts = SoftSSIIntStatus(&g_sSSI, true);

    //
    // Clear the asserted interrupt sources.
    //
    SoftSSIIntClear(&g_sSSI, ulInts);

    //
    // Handle the asserted interrupts.
    //
    ...
}

//
// Setup the software SSI and send some data.
//
void
TestSoftSSI(void)
{
    //
    // Clear the software SSI instance data.
    //
    memset(&g_sSSI, 0, sizeof(g_sSSI));

    //
    // Set the callback function used for this software SSI.
    //
    SoftSSICallbackSet(&g_sSSI, SSICallback);

    //
    // Configure the pins used for the software SSI.  This example uses
    // pins PD0, PE1, and PF2.
    //
    SoftSSIFssGPIOSet(&g_sSSI, GPIO_PORTD_BASE, GPIO_PIN_0);
    SoftSSIClkGPIOSet(&g_sSSI, GPIO_PORTE_BASE, GPIO_PIN_1);
    SoftSSITxGPIOSet(&g_sSSI, GPIO_PORTF_BASE, GPIO_PIN_2);

    //
    // Configure the data buffer used as the transmit FIFO.
    //
```

```
    SoftSSITxBufferSet(&g_sSSI, g_pusTxBuffer, 8);

    //
    // Enable the GPIO modules that contains the GPIO pins to be used by
    // the software SSI.
    //
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOD);
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF);

    //
    // Initalize the software SSI module, using mode 3 and 8 data bits.
    //
    SoftSSIConfigSet(&g_sSSI, SOFTSSI_FRF_MOTO_MODE_3, 8);

    //
    // Enable the software SSI.
    //
    SoftSSIEnable(&g_sSSI);

    //
    // Configure the timer used to generate the timing for the software
    // SSI.  The interface will be run at 10 KHz, requiring a timer tick
    // at 20 KHz.
    //
    SysCtlPeripheralEnable(SYSCTL_PERIPH_TIMER0);
    TimerConfigure(TIMER0_BASE, TIMER_CFG_32_BIT_PER);
    TimerLoadSet(TIMER0_BASE, TIMER_A, SysCtlClockGet() / 20000);
    TimerIntEnable(TIMER0_BASE, TIMER_TIMA_TIMEOUT);
    IntEnable(INT_TIMER0A);
    TimerEnable(TIMER0_BASE, TIMER_A);

    //
    // Enable the transmit FIFO half full interrupt in the software SSI.
    //
    SoftSSIIntEnable(&g_sSSI, SOFTSSI_TXFF);

    //
    // Write some data into the software SSI transmit FIFO.
    //
    SoftSSIDataPut(&g_sSSI, 0x55);
    SoftSSIDataPut(&g_sSSI, 0xaa);
    SoftSSIDataPut(&g_sSSI, 0x55);
    SoftSSIDataPut(&g_sSSI, 0xaa);
    SoftSSIDataPut(&g_sSSI, 0x55);
    SoftSSIDataPut(&g_sSSI, 0xaa);

    //
    // Wait until the software SSI is idle.  The transmit FIFO half full
    // interrupt will be sent to the callback function prior to exiting
    // this loop.
    //
    while(SoftSSIBusy(&g_sSSI))
    {
    }
}
```

As a comparison, the following is the equivalent code using the hardware SSI module and the Stellaris Peripheral Driver Library.

```
//
// The interrupt handler for the hardware SSI.
//
void
SSI0IntHandler(void)
```

```
{
    unsigned long ulInts;

    //
    // Read the asserted interrupt sources.
    //
    ulInts = SSIIntStatus(SSI0_BASE, true);

    //
    // Clear the asserted interrupt sources.
    //
    SSIIntClear(SSI0_BASE, ulInts);

    //
    // Handle the asserted interrupts.
    //
    ...
}

//
// Setup the hardware SSI and send some data.
//
void
TestSSI(void)
{
    //
    // Enable the GPIO module that contains the GPIO pins to be used by
    // the SSI, as well as the SSI module.
    //
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);
    SysCtlPeripheralEnable(SYSCTL_PERIPH_SSI0);

    //
    // Configure the GPIO pins for use by the SSI module.
    //
    GPIOPinTypeSSI(GPIO_PORTA_BASE, (GPIO_PIN_2 | GPIO_PIN_3 |
                                     GPIO_PIN_4 | GPIO_PIN_5));

    //
    // Initalize the hardware SSI module, using mode 3 and 8 data bits.
    //
    SSIConfigSetExpClk(SSI0_BASE, SysCtlClockGet(), SSI_FRF_MOTO_MODE_3,
                       SSI_MODE_MASTER, 10000, 8);

    //
    // Enable the hardware SSI.
    //
    SSIEnable(SSI0_BASE);

    //
    // Enable the transmit FIFO half full interrupt in the hardware SSI.
    //
    SSIIntEnable(SSI0_BASE, SSI_TXFF);
    IntEnable(INT_SSI0);

    //
    // Write some data into the hardware SSI transmit FIFO.
    //
    SSIDataPut(SSI0_BASE, 0x55);
    SSIDataPut(SSI0_BASE, 0xaa);
    SSIDataPut(SSI0_BASE, 0x55);
    SSIDataPut(SSI0_BASE, 0xaa);
    SSIDataPut(SSI0_BASE, 0x55);
    SSIDataPut(SSI0_BASE, 0xaa);

    //
```

```
    // Wait until the hardware SSI is idle.  The transmit FIFO half full
    // interrupt will be sent to the interrupt handler prior to exiting
    // this loop.
    //
    while(SSIBusy(SSI0_BASE))
    {
    }
}
```

13 Micro Standard Library Module

Introduction 71
API Functions 71
Programming Example 77

13.1 Introduction

The micro standard library module provides a set of small implementations of functions normally found in the C library. These functions provide reduced or greatly reduced functionality in order to remain small while still being useful for most embedded applications.

The following functions are provided, along with the C library equivalent:

Function	C library equivalent
usprintf	sprintf
usnprintf	snprintf
uvsnprintf	vsnprintf
ustrnicmp	strnicmp
ustrtoul	strtoul
ustrstr	strstr
ulocaltime	localtime

This module is contained in `utils/ustdlib.c`, with `utils/ustdlib.h` containing the API definitions for use by applications.

13.2 API Functions

Data Structures

- tTime

Functions

- void ulocaltime (unsigned long ulTime, tTime *psTime)
- int usnprintf (char *pcBuf, unsigned long ulSize, const char *pcString,...)
- int usprintf (char *pcBuf, const char *pcString,...)
- int ustrcasecmp (const char *pcStr1, const char *pcStr2)
- int ustrnicmp (const char *pcStr1, const char *pcStr2, int iCount)
- char * ustrstr (const char *pcHaystack, const char *pcNeedle)
- unsigned long ustrtoul (const char *pcStr, const char **ppcStrRet, int iBase)
- int uvsnprintf (char *pcBuf, unsigned long ulSize, const char *pcString, va_list vaArgP)

13.2.1 Data Structure Documentation

13.2.1.1 tTime

Definition:

```
typedef struct
{
    unsigned short usYear;
    unsigned char ucMon;
    unsigned char ucMday;
    unsigned char ucWday;
    unsigned char ucHour;
    unsigned char ucMin;
    unsigned char ucSec;
}
tTime
```

Members:

usYear The number of years since 0 AD.

ucMon The month, where January is 0 and December is 11.

ucMday The day of the month.

ucWday The day of the week, where Sunday is 0 and Saturday is 6.

ucHour The number of hours.

ucMin The number of minutes.

ucSec The number of seconds.

Description:

A structure that contains the broken down date and time.

13.2.2 Function Documentation

13.2.2.1 ulocaltime

Converts from seconds to calendar date and time.

Prototype:

```
void
ulocaltime(unsigned long ulTime,
           tTime *psTime)
```

Parameters:

ulTime is the number of seconds.

psTime is a pointer to the time structure that is filled in with the broken down date and time.

Description:

This function converts a number of seconds since midnight GMT on January 1, 1970 (traditional Unix epoch) into the equivalent month, day, year, hours, minutes, and seconds representation.

Returns:

None.

13.2.2.2 usnprintf

A simple snprintf function supporting %c, %d, %p, %s, %u, %x, and %X.

Prototype:

```
int
usnprintf(char *pcBuf,
          unsigned long ulSize,
          const char *pcString,
          ...)
```

Parameters:

pcBuf is the buffer where the converted string is stored.

ulSize is the size of the buffer.

pcString is the format string.

... are the optional arguments, which depend on the contents of the format string.

Description:

This function is very similar to the C library `sprintf()` function. Only the following formatting characters are supported:

- %c to print a character
- %d to print a decimal value
- %s to print a string
- %u to print an unsigned decimal value
- %x to print a hexadecimal value using lower case letters
- %X to print a hexadecimal value using lower case letters (not upper case letters as would typically be used)
- %p to print a pointer as a hexadecimal value
- %% to print out a % character

For %d, %p, %s, %u, %x, and %X, an optional number may reside between the % and the format character, which specifies the minimum number of characters to use for that value; if preceded by a 0 then the extra characters will be filled with zeros instead of spaces. For example, “%8d” will use eight characters to print the decimal value with spaces added to reach eight; “%08d” will use eight characters as well but will add zeros instead of spaces.

The type of the arguments after *pcString* must match the requirements of the format string. For example, if an integer was passed where a string was expected, an error of some kind will most likely occur.

The function will copy at most *ulSize* - 1 characters into the buffer *pcBuf*. One space is reserved in the buffer for the null termination character.

The function will return the number of characters that would be converted as if there were no limit on the buffer size. Therefore it is possible for the function to return a count that is greater than the specified buffer size. If this happens, it means that the output was truncated.

Returns:

Returns the number of characters that were to be stored, not including the NULL termination character, regardless of space in the buffer.

13.2.2.3 usprintf

A simple sprintf function supporting %c, %d, %p, %s, %u, %x, and %X.

Prototype:

```
int
usprintf(char *pcBuf,
         const char *pcString,
         ...)
```

Parameters:

pcBuf is the buffer where the converted string is stored.

pcString is the format string.

... are the optional arguments, which depend on the contents of the format string.

Description:

This function is very similar to the C library `sprintf()` function. Only the following formatting characters are supported:

- %c to print a character
- %d to print a decimal value
- %s to print a string
- %u to print an unsigned decimal value
- %x to print a hexadecimal value using lower case letters
- %X to print a hexadecimal value using lower case letters (not upper case letters as would typically be used)
- %p to print a pointer as a hexadecimal value
- %% to print out a % character

For %d, %p, %s, %u, %x, and %X, an optional number may reside between the % and the format character, which specifies the minimum number of characters to use for that value; if preceded by a 0 then the extra characters will be filled with zeros instead of spaces. For example, “%8d” will use eight characters to print the decimal value with spaces added to reach eight; “%08d” will use eight characters as well but will add zeros instead of spaces.

The type of the arguments after *pcString* must match the requirements of the format string. For example, if an integer was passed where a string was expected, an error of some kind will most likely occur.

The caller must ensure that the buffer *pcBuf* is large enough to hold the entire converted string, including the null termination character.

Returns:

Returns the count of characters that were written to the output buffer, not including the NULL termination character.

13.2.2.4 ustrcasecmp

Compares two strings without regard to case.

Prototype:

```
int
ustrcasecmp(const char *pcStr1,
            const char *pcStr2)
```

Parameters:

pcStr1 points to the first string to be compared.

pcStr2 points to the second string to be compared.

Description:

This function is very similar to the C library `strcasecmp()` function. It compares two strings without regard to case. The comparison ends if a terminating NULL character is found in either string. In this case, the shorter string is deemed the lesser.

Returns:

Returns 0 if the two strings are equal, -1 if *pcStr1* is less than *pcStr2* and 1 if *pcStr1* is greater than *pcStr2*.

13.2.2.5 ustrnicmp

Compares two strings without regard to case.

Prototype:

```
int
ustrnicmp(const char *pcStr1,
          const char *pcStr2,
          int iCount)
```

Parameters:

pcStr1 points to the first string to be compared.

pcStr2 points to the second string to be compared.

iCount is the maximum number of characters to compare.

Description:

This function is very similar to the C library `strnicmp()` function. It compares at most *iCount* characters of two strings without regard to case. The comparison ends if a terminating NULL character is found in either string before *iCount* characters are compared. In this case, the shorter string is deemed the lesser.

Returns:

Returns 0 if the two strings are equal, -1 if *pcStr1* is less than *pcStr2* and 1 if *pcStr1* is greater than *pcStr2*.

13.2.2.6 ustrstr

Finds a substring within a string.

Prototype:

```
char *
ustrstr(const char *pcHaystack,
        const char *pcNeedle)
```

Parameters:

pcHaystack is a pointer to the string that will be searched.

pcNeedle is a pointer to the substring that is to be found within *pcHaystack*.

Description:

This function is very similar to the C library `strstr()` function. It scans a string for the first instance of a given substring and returns a pointer to that substring. If the substring cannot be found, a NULL pointer is returned.

Returns:

Returns a pointer to the first occurrence of *pcNeedle* within *pcHaystack* or NULL if no match is found.

13.2.2.7 ustrtoul

Converts a string into its numeric equivalent.

Prototype:

```
unsigned long
ustrtoul(const char *pcStr,
         const char **ppcStrRet,
         int iBase)
```

Parameters:

pcStr is a pointer to the string containing the integer.

ppcStrRet is a pointer that will be set to the first character past the integer in the string.

iBase is the radix to use for the conversion; can be zero to auto-select the radix or between 2 and 16 to explicitly specify the radix.

Description:

This function is very similar to the C library `strtoul()` function. It scans a string for the first token (that is, non-white space) and converts the value at that location in the string into an integer value.

Returns:

Returns the result of the conversion.

13.2.2.8 uvsnprintf

A simple vsnprintf function supporting %c, %d, %p, %s, %u, %x, and %X.

Prototype:

```
int
uvsnprintf(char *pcBuf,
           unsigned long ulSize,
           const char *pcString,
           va_list vaArgP)
```

Parameters:

pcBuf points to the buffer where the converted string is stored.

ulSize is the size of the buffer.

pcString is the format string.

vaArgP is the list of optional arguments, which depend on the contents of the format string.

Description:

This function is very similar to the C library `vsnprintf()` function. Only the following formatting characters are supported:

- %c to print a character
- %d to print a decimal value
- %s to print a string
- %u to print an unsigned decimal value
- %x to print a hexadecimal value using lower case letters
- %X to print a hexadecimal value using lower case letters (not upper case letters as would typically be used)
- %p to print a pointer as a hexadecimal value
- %% to print out a % character

For %d, %p, %s, %u, %x, and %X, an optional number may reside between the % and the format character, which specifies the minimum number of characters to use for that value; if preceded by a 0 then the extra characters will be filled with zeros instead of spaces. For example, “%8d” will use eight characters to print the decimal value with spaces added to reach eight; “%08d” will use eight characters as well but will add zeroes instead of spaces.

The type of the arguments after *pcString* must match the requirements of the format string. For example, if an integer was passed where a string was expected, an error of some kind will most likely occur.

The *ulSize* parameter limits the number of characters that will be stored in the buffer pointed to by *pcBuf* to prevent the possibility of a buffer overflow. The buffer size should be large enough to hold the expected converted output string, including the null termination character.

The function will return the number of characters that would be converted as if there were no limit on the buffer size. Therefore it is possible for the function to return a count that is greater than the specified buffer size. If this happens, it means that the output was truncated.

Returns:

Returns the number of characters that were to be stored, not including the NULL termination character, regardless of space in the buffer.

13.3 Programming Example

The following example shows how to use some of the micro standard library functions.

```
unsigned long ulValue;
char pcBuffer[32];
tTime sTime;

//
// Convert the number in pcBuffer (previous read from somewhere) into an
// integer.  Note that this supports converting decimal values (such as
// 4583), octal values (such as 036583), and hexadecimal values (such as
// 0x3425).
```

```
//
ulValue = ustrtoul(pcBuffer, 0, 0);

//
// Convert that integer from a number of seconds into a broken down date.
//
ulocaltime(ulValue, &sTime);

//
// Print out the corresponding time of day in military format.
//
usprintf(pcBuffer, "%02d:%02d", sTime.ucHour, sTime.ucMin);
```

14 UART Standard IO Module

Introduction 79
API Functions 80
Programming Example 86

14.1 Introduction

The UART standard IO module provides a simple interface to a UART that is similar to the standard IO package available in the C library. Only a very small subset of the normal functions are provided; UARTprintf() is an equivalent to the C library printf() function and UARTgets() is an equivalent to the C library fgets() function.

This module is contained in `utils/uartstdio.c`, with `utils/uartstdio.h` containing the API definitions for use by applications.

14.1.1 Unbuffered Operation

Unbuffered operation is selected by not defining **UART_BUFFERED** when building the UART standard IO module. In unbuffered mode, calls to the module will not return until the operation has been completed. So, for example, a call to UARTprintf() will not return until the entire string has be placed into the UART's FIFO. If it is not possible for the function to complete its operation immediately, it will busy wait.

14.1.2 Buffered Operation

Buffered operation is selected by defining **UART_BUFFERED** when building the UART standard IO module. In buffered mode, there is a larger UART data FIFO in SRAM that extends the size of the hardware FIFO. Interrupts from the UART are used to transfer data between the SRAM buffer and the hardware FIFO. It is the responsibility of the application to ensure that UARTStdioIntHandler() is called when the UART interrupt occurs; typically this is accomplished by placing it in the vector table in the startup code for the application.

In addition providing a larger UART buffer, the behavior of UARTprintf() is slightly modified. If the output buffer is full, UARTprintf() will discard the remaining characters from the string instead of waiting until space becomes available in the buffer. If this behavior is not desired, UARTFlushTx() may be called to ensure that the transmit buffer is emptied prior to adding new data via UARTprintf() (though this will not work if the string to be printed is larger than the buffer).

UARTPeek() can be used to determine whether a line end is present prior to calling UARTgets() if non-blocking operation is required. In cases where the buffer supplied on UARTgets() fills before a line termination character is received, the call will return with a full buffer.

14.2 API Functions

Functions

- void UARTEchoSet (tBoolean bEnable)
- void UARTFlushRx (void)
- void UARTFlushTx (tBoolean bDiscard)
- unsigned char UARTgetc (void)
- int UARTgets (char *pcBuf, unsigned long ulLen)
- int UARTPeek (unsigned char ucChar)
- void UARTprintf (const char *pcString,...)
- int UARTRxBytesAvail (void)
- void UARTStdioInit (unsigned long ulPortNum)
- void UARTStdioInitExpClk (unsigned long ulPortNum, unsigned long ulBaud)
- void UARTStdioIntHandler (void)
- int UARTTxBytesFree (void)
- int UARTwrite (const char *pcBuf, unsigned long ulLen)

14.2.1 Function Documentation

14.2.1.1 UARTEchoSet

Enables or disables echoing of received characters to the transmitter.

Prototype:

```
void
UARTEchoSet(tBoolean bEnable)
```

Parameters:

bEnable must be set to **true** to enable echo or **false** to disable it.

Description:

This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to control whether or not received characters are automatically echoed back to the transmitter. By default, echo is enabled and this is typically the desired behavior if the module is being used to support a serial command line. In applications where this module is being used to provide a convenient, buffered serial interface over which application-specific binary protocols are being run, however, echo may be undesirable and this function can be used to disable it.

Returns:

None.

14.2.1.2 UARTFlushRx

Flushes the receive buffer.

Prototype:

```
void
UARTFlushRx(void)
```

Description:

This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to discard any data received from the UART but not yet read using UARTgets().

Returns:

None.

14.2.1.3 UARTFlushTx

Flushes the transmit buffer.

Prototype:

```
void
UARTFlushTx(tBoolean bDiscard)
```

Parameters:

bDiscard indicates whether any remaining data in the buffer should be discarded (**true**) or transmitted (**false**).

Description:

This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to flush the transmit buffer, either discarding or transmitting any data received via calls to UARTprintf() that is waiting to be transmitted. On return, the transmit buffer will be empty.

Returns:

None.

14.2.1.4 UARTgetc

Read a single character from the UART, blocking if necessary.

Prototype:

```
unsigned char
UARTgetc(void)
```

Description:

This function will receive a single character from the UART and store it at the supplied address.

In both buffered and unbuffered modes, this function will block until a character is received. If non-blocking operation is required in buffered mode, a call to UARTRxAvail() may be made to determine whether any characters are currently available for reading.

Returns:

Returns the character read.

14.2.1.5 UARTgets

A simple UART based get string function, with some line processing.

Prototype:

```
int
UARTgets(char *pcBuf,
         unsigned long ulLen)
```

Parameters:

pcBuf points to a buffer for the incoming string from the UART.

ulLen is the length of the buffer for storage of the string, including the trailing 0.

Description:

This function will receive a string from the UART input and store the characters in the buffer pointed to by *pcBuf*. The characters will continue to be stored until a termination character is received. The termination characters are CR, LF, or ESC. A CRLF pair is treated as a single termination character. The termination characters are not stored in the string. The string will be terminated with a 0 and the function will return.

In both buffered and unbuffered modes, this function will block until a termination character is received. If non-blocking operation is required in buffered mode, a call to UARTPeek() may be made to determine whether a termination character already exists in the receive buffer prior to calling UARTgets().

Since the string will be null terminated, the user must ensure that the buffer is sized to allow for the additional null character.

Returns:

Returns the count of characters that were stored, not including the trailing 0.

14.2.1.6 UARTPeek

Looks ahead in the receive buffer for a particular character.

Prototype:

```
int
UARTPeek(unsigned char ucChar)
```

Parameters:

ucChar is the character that is to be searched for.

Description:

This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to look ahead in the receive buffer for a particular character and report its position if found. It is typically used to determine whether a complete line of user input is available, in which case ucChar should be set to CR ('\r') which is used as the line end marker in the receive buffer.

Returns:

Returns -1 to indicate that the requested character does not exist in the receive buffer. Returns a non-negative number if the character was found in which case the value represents the position of the first instance of *ucChar* relative to the receive buffer read pointer.

14.2.1.7 UARTprintf

A simple UART based printf function supporting %c, %d, %p, %s, %u, %x, and %X.

Prototype:

```
void
UARTprintf(const char *pcString,
           ...)
```

Parameters:

pcString is the format string.

... are the optional arguments, which depend on the contents of the format string.

Description:

This function is very similar to the C library `fprintf()` function. All of its output will be sent to the UART. Only the following formatting characters are supported:

- %c to print a character
- %d to print a decimal value
- %s to print a string
- %u to print an unsigned decimal value
- %x to print a hexadecimal value using lower case letters
- %X to print a hexadecimal value using lower case letters (not upper case letters as would typically be used)
- %p to print a pointer as a hexadecimal value
- %% to print out a % character

For %s, %d, %u, %p, %x, and %X, an optional number may reside between the % and the format character, which specifies the minimum number of characters to use for that value; if preceded by a 0 then the extra characters will be filled with zeros instead of spaces. For example, “%8d” will use eight characters to print the decimal value with spaces added to reach eight; “%08d” will use eight characters as well but will add zeroes instead of spaces.

The type of the arguments after *pcString* must match the requirements of the format string. For example, if an integer was passed where a string was expected, an error of some kind will most likely occur.

Returns:

None.

14.2.1.8 UARTRxBytesAvail

Returns the number of bytes available in the receive buffer.

Prototype:

```
int
UARTRxBytesAvail(void)
```

Description:

This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to determine the number of bytes of data currently available in the receive buffer.

Returns:
Returns the number of available bytes.

14.2.1.9 UARTStdioInit

Initializes the UART console.

Prototype:
```
void
UARTStdioInit(unsigned long ulPortNum)
```

Parameters:
ulPortNum is the number of UART port to use for the serial console (0-2)

Description:
This function will initialize the specified serial port to be used as a serial console. The serial parameters will be set to 115200, 8-N-1. An application wishing to use a different baud rate may call UARTStdioInitExpClk() instead of this function.

This function or UARTStdioInitExpClk() must be called prior to using any of the other UART console functions: UARTprintf() or UARTgets(). In order for this function to work correctly, SysCtlClockSet() must be called prior to calling this function.

It is assumed that the caller has previously configured the relevant UART pins for operation as a UART rather than as GPIOs.

Returns:
None.

14.2.1.10 UARTStdioInitExpClk

Initializes the UART console and allows the baud rate to be selected.

Prototype:
```
void
UARTStdioInitExpClk(unsigned long ulPortNum,
                    unsigned long ulBaud)
```

Parameters:
ulPortNum is the number of UART port to use for the serial console (0-2)
ulBaud is the bit rate that the UART is to be configured to use.

Description:
This function will initialize the specified serial port to be used as a serial console. The serial parameters will be set to 8-N-1 and the bit rate set according to the value of the *ulBaud* parameter.

This function or UARTStdioInit() must be called prior to using any of the other UART console functions: UARTprintf() or UARTgets(). In order for this function to work correctly, SysCtlClockSet() must be called prior to calling this function. An application wishing to use 115,200 baud may call UARTStdioInit() instead of this function but should not call both functions.

It is assumed that the caller has previously configured the relevant UART pins for operation as a UART rather than as GPIOs.

Returns:
None.

14.2.1.11 UARTStdioIntHandler

Handles UART interrupts.

Prototype:
```
void
UARTStdioIntHandler(void)
```

Description:
This function handles interrupts from the UART. It will copy data from the transmit buffer to the UART transmit FIFO if space is available, and it will copy data from the UART receive FIFO to the receive buffer if data is available.

Returns:
None.

14.2.1.12 UARTTxBytesFree

Returns the number of bytes free in the transmit buffer.

Prototype:
```
int
UARTTxBytesFree(void)
```

Description:
This function, available only when the module is built to operate in buffered mode using **UART_BUFFERED**, may be used to determine the amount of space currently available in the transmit buffer.

Returns:
Returns the number of free bytes.

14.2.1.13 UARTwrite

Writes a string of characters to the UART output.

Prototype:
```
int
UARTwrite(const char *pcBuf,
          unsigned long ulLen)
```

Parameters:
pcBuf points to a buffer containing the string to transmit.
ulLen is the length of the string to transmit.

Description:

This function will transmit the string to the UART output. The number of characters transmitted is determined by the *ulLen* parameter. This function does no interpretation or translation of any characters. Since the output is sent to a UART, any LF (/n) characters encountered will be replaced with a CRLF pair.

Besides using the *ulLen* parameter to stop transmitting the string, if a null character (0) is encountered, then no more characters will be transmitted and the function will return.

In non-buffered mode, this function is blocking and will not return until all the characters have been written to the output FIFO. In buffered mode, the characters are written to the UART transmit buffer and the call returns immediately. If insufficient space remains in the transmit buffer, additional characters are discarded.

Returns:

Returns the count of characters written.

14.3 Programming Example

The following example shows how to use the UART standard IO module to write a string to the UART “console”.

```
//
// Configure the appropriate pins as UART pins; in this case, PA0/PA1 are
// used for UART0.
//
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);
GPIOPinTypeUART(GPIO_PORTA_BASE, GPIO_PIN_0 | GPIO_PIN_1);

//
// Initialize the UART standard IO module.
//
UARTStdioInit(0);

//
// Print a string.
//
UARTprintf("Hello world!\n");
```

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

TI products are not authorized for use in safety-critical applications (such as life support) where a failure of the TI product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Buyers represent that they have all necessary expertise in the safety and regulatory ramifications of their applications, and acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of TI products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by TI. Further, Buyers must fully indemnify TI and its representatives against any damages arising out of the use of TI products in such safety-critical applications.

TI products are neither designed nor intended for use in military/aerospace applications or environments unless the TI products are specifically designated by TI as military-grade or "enhanced plastic." Only products designated by TI as military-grade meet military specifications. Buyers acknowledge and agree that any such use of TI products which TI has not designated as military-grade is solely at the Buyer's risk, and that they are solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI products are neither designed nor intended for use in automotive applications or environments unless the specific TI products are designated by TI as compliant with ISO/TS 16949 requirements. Buyers acknowledge and agree that, if they use any non-designated products in automotive applications, TI will not be responsible for any failure to meet such requirements.

Following are URLs where you can obtain information on other Texas Instruments products and application solutions:

Products		**Applications**	
Amplifiers	amplifier.ti.com	Audio	www.ti.com/audio
Data Converters	dataconverter.ti.com	Automotive	www.ti.com/automotive
DLP® Products	www.dlp.com	Broadband	www.ti.com/broadband
DSP	dsp.ti.com	Digital Control	www.ti.com/digitalcontrol
Clocks and Timers	www.ti.com/clocks	Medical	www.ti.com/medical
Interface	interface.ti.com	Military	www.ti.com/military
Logic	logic.ti.com	Optical Networking	www.ti.com/opticalnetwork
Power Mgmt	power.ti.com	Security	www.ti.com/security
Microcontrollers	microcontroller.ti.com	Telephony	www.ti.com/telephony
RFID	www.ti-rfid.com	Video & Imaging	www.ti.com/video
RF/IF and ZigBee® Solutions	www.ti.com/lprf	Wireless	www.ti.com/wireless

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2006-2010, Texas Instruments Incorporated